International Journal of Information Security (2026) 25:176
https://doi.org/10.1007/s10207-026-01340-z

REGULAR CONTRIBUTION

Multi-stage attack detection in IIoT supply chains using graph neural networks

Williams Afrifah[1] · Gregory Epiphaniou[1,2] · Harjinder Singh Lallie[1] · Carsten Maple[1]

Received: 7 February 2026 / Accepted: 21 September 2026
© The Author(s) 2026

Abstract

Industrial internet of things (IIoT) environments connect devices, gateways, hosts, and services in ways that increase both operational visibility and cyber-attack exposure. Conventional intrusion-detection systems commonly produce flat attack labels and may underuse relational information between communicating assets. We investigate an ATT&CK-inspired stage-aware intrusion-detection framework that maps observed behaviour into four operational classes: Benign, Initial Access & Discovery (IAD), Lateral Movement, Execution & Persistence (LMEP), and Impact (IMP). Network-flow observations are represented using leakage-aware feature processing and an asset–time interaction graph, enabling comparison of feature-based, graph-based, temporal, and fusion models. To reduce dependence on a single benchmark, we evaluate the stage-aware formulation on both ToN-IoT and Edge-IIoTset. Following correction of the temporal-windowing pipeline, ToN-IoT contains 34,435 asset–window instances. GAT achieves the strongest ToN-IoT stage-detection performance with Macro-F1 0.687 ± 0.041, followed by GraphSAGE (0.671 ± 0.015) and GCN (0.660 ± 0.039). The logistic-regression RF–GCN stack reaches 0.395 ± 0.067 and does not outperform the strongest graph learners. A controlled meta-learner ablation increases stacked Macro-F1 to 0.468 with gradient boosting, suggesting that meta-learner choice contributes substantially to the fusion shortfall. On Edge-IIoTset, the feature-based Random Forest performs best with Macro-F1 0.615. We additionally evaluate stage-mapping sensitivity, graph construction, window-size sensitivity, alternative stacking meta-learners, baseline tuning, bootstrap uncertainty, and inference latency. We formulate a strict leakage-controlled pre-impact forecasting protocol; however, although the corrected ToN-IoT representation yields 21,313 candidate forecasting instances, only five are positive and all occur in the training partition, with zero positives in validation and test. We therefore do not report held-out forecasting-performance estimates and limit our empirical claims to stage-aware intrusion detection.

Keywords Industrial internet-of-things · Supply chain security · Intrusion detection system (IDS) · Graph neural networks · Multi-stage attack detection · ATT& CK- inspired staging

1 Introduction

Industrial supply chains are undergoing rapid digital transformation driven by Industrial Internet of Things (IIoT) technologies deployed across manufacturing plants, warehouses, logistics fleets, and retail hubs [1]. Sensors, actuators, programmable logic controllers (PLCs), warehouse management systems (WMS), transport tracking devices, and cloud analytics platforms are increasingly interconnected across organisational and geographic boundaries to enable real-time monitoring, optimisation, and automation [2–4]. While this connectivity improves visibility and efficiency, it also expands the cyber attack surface by introducing new dependencies among IT systems, operational technology (OT)

✉ Williams Afrifah
williams.g.afrifah@warwick.ac.uk

Gregory Epiphaniou
gregory.epiphaniou@warwick.ac.uk;
gregory.epiphaniou@kfupm.edu.sa

Harjinder Singh Lallie
HL@warwick.ac.uk

Carsten Maple
CM@warwick.ac.uk

[1] Secure Cyber Systems Group, WMG, University of Warwick, Coventry CV4 7AL, UK

[2] Information and Computer Science Department, King Fahd University of Petroleum and Minerals, Dhahran, Saudi Arabia

Published online: 16 October 2026

assets, and third-party services along the extended value chain [5].

This expanded attack surface has been exploited in practice. Adversaries increasingly leverage supply-chain relationships and embedded IoT/IIoT components as entry points to compromise upstream or downstream partners, disrupt production, manipulate inventory, and interfere with logistics operations [2, 3, 6, 7]. In such environments, intrusions often unfold as multi-stage campaigns: attackers establish an initial foothold (e.g., through exposed services or credential abuse), perform discovery and propagation across integration points (e.g., VPNs, APIs, shared cloud services), and eventually execute high-impact actions such as denial-of-service or operational manipulation. Detecting these campaigns is challenging because evidence is distributed across heterogeneous telemetry sources (network flows, IIoT sensor streams, and host logs), and because stage progression is inherently relational and temporal: early events that appear benign in isolation may become suspicious when correlated across assets and time.

Intrusion Detection Systems (IDSs) remain a core defensive control, but many data-driven IDS pipelines for IoT/IIoT environments still provide limited operational support for campaign-aware triage. First, detection is often formulated as flat classification (benign vs. malicious or attack-type labels), which does not explicitly model progression toward high-severity outcomes [8–10]. Second, many pipelines underutilise cross-asset context: they score events independently and therefore miss relational patterns (e.g., shared targets, repeated authentication failures, or coordinated probing across subnets) that are characteristic of multi-stage intrusions. Third, although structured adversary knowledge bases such as MITRE ATT&CK, ATT&CK for Industrial Control Systems (ICS) and ICSThreatQA are widely used for threat modelling and defensive planning [11–13], ATT&CK-inspired staging is not commonly operationalised inside end-to-end, data-driven IDS pipelines, despite its potential to improve interpretability and response workflows [14, 15].

Graph Neural Networks (GNNs) offer a principled way to model relational structure for intrusion detection by representing events and entities as graphs and learning from both node features and neighbourhood dependencies [16–20]. However, most GNN-based IDS studies focus primarily on network traffic within a single administrative domain, and many do not fuse multi-source telemetry typical of IIoT supply-chain settings nor provide stage-aware outputs aligned with an adversary lifecycle. Moreover, the majority of IDS work remains reactive: it detects malicious activity after it is underway, with comparatively less emphasis on forecasting whether early-stage signals are likely to escalate into operational impact.

The ToN-IoT family of datasets provides an opportunity to study these issues in a controlled but heterogeneous setting by offering multi-modal telemetry collected from IIoT testbeds, including network traffic, IoT sensor data, and Linux/Windows host logs [21–24]. While ToN-IoT has been widely used to benchmark IDS models, most prior evaluations consider modalities in isolation or focus on flat attack detection, leaving open the problem of stage-aware intrusion detection that explicitly uses cross-asset relational context and provides operationally interpretable campaign-stage outputs.

Research objective Our objective is to investigate whether ATT&CK-inspired operational staging and relational graph context can improve the interpretability and generalisation of IIoT intrusion detection across independently collected cybersecurity benchmarks. We additionally formulate a leakage-controlled pre-impact forecasting protocol, while treating its empirical validation separately from the stage-detection task.

1.1 Motivation and research gap

Motivated by this objective, we identify three gaps in existing data-driven IDS pipelines for IIoT-like environments:

- *G1: Lack of stage-aware, interpretable outputs.* Many IDS approaches operate with flat labels and do not explicitly model campaign progression. Stage-aware outputs aligned with an adversary lifecycle can support triage and response by indicating *where* an alert sits in an intrusion sequence.
- *G2: Limited relational reasoning across communicating assets.*Many IDS pipelines classify observations independently and therefore underuse communication relationships and neighbourhood context that may help distinguish reconnaissance, post-compromise activity, and disruptive behaviour.
- *G3: Limited methodology for leakage-controlled escalation forecasting.* Although forecasting progression toward operational impact is operationally attractive, such evaluation requires reliable longitudinal asset trajectories and strict exclusion of post-impact information. Existing benchmark representations do not always preserve the temporal structure required for this task.

Novelty statement The contribution is centred on three elements. *First*, we operationalise an ATT&CK-inspired stage representation that transforms dataset-specific attack labels into the interpretable classes IAD, LMEP, and IMP and explicitly evaluate the sensitivity of this abstraction to alternative mappings. *Second*, we investigate complementary feature-based and relational reasoning through RF, GCN, late fusion, and out-of-fold RF–GCN stacking under identity-preserving evaluation, including explicit ablations of graph construction and meta-learner choice. *Third*, we formulate a

strict leakage-controlled protocol for future pre-impact forecasting in which only evidence preceding a per-asset cutoff is admissible. Because the validated real-data benchmarks do not provide sufficient reliable longitudinal progression for statistically meaningful forecasting, we distinguish this methodological formulation from the empirically validated stage-detection contribution.

Scope of cross-dataset generalisation To reduce dependence on a single benchmark, we evaluate the stage-aware detection framework on two independently collected public IoT/IIoT cybersecurity datasets: ToN-IoT and Edge-IIoTset. The same four-class operational abstraction—Benign, Initial Access & Discovery (IAD), Lateral Movement, Execution & Persistence (LMEP), and Impact (IMP)—is applied to both datasets using a common deterministic stage-assignment rubric. This second-dataset evaluation allows us to examine whether the proposed stage abstraction and detection methodology remain applicable beyond the characteristics of ToN-IoT.

We distinguish, however, between cross-dataset validation of stage-aware detection and validation of temporal impact forecasting. The real ToN-IoT network dataset supports timestamp-aware stage detection and graph construction, whereas the selected Edge-IIoTset ML dataset contains incomplete calendar information in its published `frame.time` field. Moreover, under the leakage-controlled sequence-construction protocol, the available real-data asset trajectories do not provide sufficient multi-window pre-impact progression instances to support a statistically valid impact-forecasting experiment. Consequently, the revised empirical claims are limited to stage-aware intrusion detection across the two benchmarks; the pre-impact forecasting architecture is retained as a methodological formulation whose empirical validation requires datasets containing reliable longitudinal attack progression at the asset level.

1.2 Contributions

This paper makes the following contributions:

- We formulate an ATT&CK-inspired stage-aware intrusion-detection task that maps dataset-specific attacks into the operational classes Benign, IAD, LMEP, and IMP, and evaluate the formulation on both ToN-IoT and Edge-IIoTset.
- We investigate feature-based and graph-based stage detection using RF, GCN, late fusion, and asset-disjoint out-of-fold RF–GCN stacking, together with stronger tabular and graph-learning baselines.
- We evaluate key modelling assumptions through stage-mapping sensitivity, graph-construction ablations, temporal-window sensitivity where valid, alternative stacking meta-learners, baseline tuning, bootstrap uncertainty analysis, and inference-latency measurement.
- We define a leakage-controlled pre-impact forecasting protocol that excludes post-cutoff and IMP-stage evidence from predictor inputs. The revised study explicitly distinguishes this methodological formulation from empirical forecasting validation because the available benchmark trajectories do not provide sufficient valid longitudinal pre-impact sequences.

Feature-based and graph-based models provide different representations of the same stage-detection problem. Rather than assuming that combining them must improve performance, we evaluate RF, GCN, late fusion, and learned stacking empirically. The revised cross-dataset results show that their relative effectiveness depends on dataset characteristics and that a learned stacking layer can underperform the strongest base learner. This motivates reporting the constituent models and fusion strategies separately rather than treating stacking as universally superior.

The rest of the paper is organised as follows. Section 2 reviews background and related work on IIoT supply-chain security, ATT&CK-based modelling, and graph-based intrusion detection. Section 3 presents the threat model and the ATT&CK-inspired three-stage abstraction used in this work. Section 4 details the proposed multi-stage detection and impact prediction framework. Section 5 describes the experimental setup and evaluation methodology. Section 6 discusses results and error analysis. Section 7 discuses the Reproducibility and Data-Correction and Sect. 8 concludes the paper and outlines future research directions.

2 Background and related work

This section reviews background concepts and related work along four axes: (i) security of IIoT-enabled supply chains, (ii) MITRE ATT&CK and ATT&CK for ICS as knowledge bases of adversary behaviour, (iii) intrusion detection on multi-modal datasets such as ToN-IoT, and (iv) recent progress on graph-based and multi-stage intrusion detection.

2.1 IIoT-enabled supply chains and security challenges

The increasing deployment of Industrial Internet of Things (IIoT) technologies in manufacturing, logistics, and retail is transforming supply chains into tightly coupled cyber–physical systems. Sensors, actuators, PLCs, warehouse management systems (WMS), transport tracking devices, and cloud analytics platforms are interconnected across organisational and geographic boundaries to support real-time visibility, predictive maintenance, and automated decision-

making [2–4]. This connectivity, however, expands the attack surface: adversaries can target not only enterprise IT networks but also IIoT gateways, field devices, and cloud connectors, exploiting trust relationships between partners to move laterally and maximise disruption [1, 5, 6, 25].

Empirical analyses and incident reports highlight that supply-chain attacks often involve compromise of third-party remote access (e.g., VPNs), abuse of exposed APIs or cloud services, and manipulation of telemetry or process parameters in ways that degrade production, alter inventory data, or interfere with logistics planning [2, 3]. These campaigns are typically multi-stage, progressing from initial footholds to propagation across integration points and culminating in operational impact such as ransomware or disruption. Consequently, detection mechanisms that operate within a single organisation or rely on a single telemetry type can miss cross-layer patterns that characterise supply-chain intrusions.

2.2 Intrusion detection systems in IIoT

Intrusion detection in Industrial Internet of Things (IIoT) environments has evolved from the direct reuse of traditional network intrusion detection systems (NIDS) to dedicated designs tailored to industrial communication patterns, resource constraints, and safety requirements. Early deployments typically relied on signature- and rule-based engines such as Snort and Suricata [26, 27], often placed at industrial demilitarised zones (DMZs) or gateways. While such systems remain effective for detecting known exploits, they struggle with zero-day threats, protocol heterogeneity, encrypted traffic, and the stringent timing and availability demands of industrial control networks [28, 29]. These limitations have motivated a shift towards data-driven IDS approaches that leverage machine learning (ML) and deep learning (DL) to detect complex and previously unseen attacks in IIoT settings [8, 30].

A first wave of IIoT IDS research applied conventional ML classifiers to flow-level or device-level features. Surveys such as [8, 29, 31] report extensive use of decision trees, Random Forests, support vector machines, and k-nearest neighbours. These models are attractive due to their interpretability and relatively low computational cost, making them suitable for deployment at IIoT gateways or edge nodes. Building on this, several works explore ensemble-based IDS that combine multiple base learners to improve robustness [32–36].

With the rise of deep learning, a growing body of work has investigated DL architectures for IIoT intrusion detection, including fully connected and convolutional models [28, 37, 38] and approaches that integrate optimisation or reinforcement learning for adaptive detection [39, 40]. Comprehensive surveys underline persistent challenges: most systems operate on single-modality data, treat samples as independent, and remain predominantly reactive rather than predictive [28, 30, 31].

2.3 Adaptive, reinforcement-learning, and privacy-aware IIoT IDS

Recent IIoT intrusion-detection research has also investigated reinforcement learning, open-set recognition, adversarial decision making, and privacy-aware model maintenance. Deep Q-Network-based approaches have been proposed for open-set IIoT intrusion detection, where the objective is to identify both known and previously unseen attack behaviours [41]. Related DQN-based heuristic IDS approaches target zero-day attacks in edge-based Social IoT environments [42]. Other studies formulate IIoT intrusion detection using stochastic games and Double Deep Q-Networks to adapt defensive decisions under adversarial and resource constraints [43]. Multi-teacher knowledge-distillation approaches such as MT-KD have additionally been proposed for privacy-aware machine unlearning in IIoT intrusion-detection systems [44]. These studies address complementary objectives. DQN/DDQN approaches primarily focus on adaptive decision policies, open-set or zero-day recognition, and resource-aware optimisation, whereas MT-KD focuses on selective unlearning and privacy-aware model maintenance. The present work instead focuses on operational stage representation and identity-preserving feature/graph-based classification of intrusion progression.

2.4 Multi-stage and predictive attack modelling

Intrusions against complex infrastructures unfold as campaigns composed of reconnaissance, foothold establishment, lateral movement, and eventual impact. Early work addressed multi-step reasoning by correlating low-level IDS alerts into attack graphs or scenarios [45–47]. Subsequent approaches modernised these ideas with probabilistic and learning-based correlators such as Bayesian networks and hidden Markov models for real-time multistep forecasting [48, 49].

Graph neural networks (GNNs) have recently been applied to alert graphs and provenance graphs to discover attack scenarios and capture higher-order dependencies [50–52]. While these approaches demonstrate the value of graph structure for campaign reasoning, they are typically host-centric and are not designed to fuse heterogeneous IIoT supply-chain telemetry spanning devices, gateways, enterprise hosts, and network flows.

A complementary line of research investigates predictive modelling of attacks and alerts. Surveys review attack projection and forecasting in cyber security [53–55], and recurrent architectures such as GRU-based models have been explored for forecasting future alerts from historical sequences [56]. However, most predictive systems forecast future alerts or

attack steps without tightly coupling prediction to an interpretable, stage-aware detection model and without explicitly leveraging multi-modal IIoT telemetry.

2.5 Temporal and dynamic graph learning

Temporal graph neural networks provide an alternative to separating graph-based and sequence-based reasoning. DySAT combines structural and temporal self-attention over evolving graph snapshots [57]; EvolveGCN evolves graph-convolution parameters across graph sequences [58]; TGAT incorporates temporal encoding and attention for inductive temporal-graph learning [59]; and TGN provides a general memory-based framework for streams of timestamped graph interactions [60]. These approaches are particularly relevant when reliable longitudinal communication graphs are available.

Our revised study does not claim superiority over temporal GNNs. A fair comparison for pre-impact forecasting requires reliable longitudinal asset-level graph events, which are not available in sufficient form in the validated benchmark representations used here. Comparison with TGAT, TGN, DySAT, EvolveGCN, and related graph transformers is therefore identified as a concrete extension for future longitudinal IIoT datasets.

2.6 Positioning and novelty

Existing IDS approaches most relevant to IIoT-enabled supply chains can be broadly grouped into three strands. First, a large body of ML/DL IDS work trains detectors on a single modality (commonly network flows or host logs) and optimises flat classification objectives, which limits campaign-level interpretability and cross-layer correlation. Second, multi-stage intrusion reasoning is often implemented via handcrafted alert-correlation rules or probabilistic correlators that require carefully curated alert types and may not directly leverage heterogeneous raw telemetry. Third, graph-based IDS methods exploit relational structure but are frequently network-centric (or host-centric), and are not consistently organised around an explicit, lifecycle-oriented staging abstraction or evaluated as an end-to-end pipeline that also supports proactive impact forecasting.

Our framework is compatible with this architecture: the entity–window representation provides a structured analysis unit, while the interaction graph provides a learned analogue of communication-based correlation that produces stage-aligned alerts. The leakage-controlled forecasting formulation could provide an additional prioritisation layer when suitable longitudinal telemetry becomes available, but this capability is not empirically validated in the present study.

These observations motivate an IDS design that (i) represents attack progression using operationally interpretable stages, (ii) incorporates cross-asset relational context for stage-aware reasoning, and (iii) defines a leakage-controlled formulation for future escalation forecasting. In the present experiments, these principles are evaluated using network telemetry; extension to additional host and device modalities remains future work.

Explicit contrast to prior ATT&CK-based IDS and forecasting systems
Prior ATT&CK-inspired IDS work typically uses ATT&CK mainly as a taxonomy for labelling or post-hoc interpretation, but does not operationalise a fully leakage-controlled *pre-impact* forecasting task with asset-disjoint evaluation. Conversely, alert-forecasting and incident-prediction systems commonly forecast future alerts or steps from historical sequences, but they often (i) do not condition prediction on an explicit stage model, (ii) do not enforce strict pre-impact cutoffs that exclude any impact-stage evidence as input, and/or (iii) evaluate with record-level splits that can leak identity-specific baselines. Our framework couples stage-aware detection to stage-conditioned forecasting under explicit controls: asset-disjoint splits, per-asset cutoffs, and pre-t graph/sequence construction, so the impact score reflects genuinely pre-impact evidence rather than post-impact signatures (Table 1).

3 Threat model and ATT&CK-inspired staging

3.1 Threat model

We consider an adversary targeting an IIoT-enabled supply chain environment composed of heterogeneous assets, including field devices (e.g., sensors and actuators), IIoT gateways, on-premise application servers, and cloud-hosted services that support logistics, planning, and asset tracking. The adversary's objective is to degrade, disrupt, or manipulate supply-chain operations, for example, by impairing service availability, falsifying operational data, or interfering with monitoring and decision-making.

Adversary capabilities We assume the attacker can conduct network-based activity (e.g., scanning and credential attacks), exploit exposed services, and abuse compromised third-party or supplier-maintained infrastructure. The attacker may establish persistence, misuse credentials, and move laterally across reachable components and integration points. We do not assume physical access to industrial assets.

Defender visibility and constraints The defender collects heterogeneous telemetry, including network flow records, host-level logs from Linux/Windows systems, and IIoT

Table 1 Capability-oriented positioning of the revised framework. The empirical contribution focuses on ATT&CK-inspired stage-aware detection; pre-impact forecasting is formulated but not empirically validated on the available longitudinal benchmark representations

Approach family	Multi-modal	Graph-based	ATT&CK-aligned	Multi-stage	Predictive
Single-modality feature-based IDS (typical ML/DL)	sometimes	×	×	×	×
Graph-based IDS (traffic-/host-centric)	×	✓	×	×	×
Stage-aware or campaign reasoning (correlation / HMM / rules)	sometimes	sometimes	partial	✓	sometimes
Alert/incident forecasting (sequence models)	sometimes	×	×	sometimes	✓
This work	Network telemetry	✓	✓	✓	formulated, not empirically validated

device telemetry. Due to device resource constraints, organisational boundaries, and collection gaps, telemetry may be incomplete or noisy. The proposed framework is designed to operate under imperfect visibility by combining feature-level evidence with relational context captured in an *asset–time window interaction graph*.

3.2 Stage definition and label semantics

Motivated by ATT&CK for ICS and the typical progression of supply-chain intrusions and Kumar et al.'s approach [61, 62], we abstract adversary behaviour into three operational stages that support both interpretability and prediction:

- IAD (initial access and discovery). Actions used to obtain an initial foothold and characterise the environment, such as scanning, probing exposed services, brute-force authentication attempts, and reconnaissance of devices and services.
- LMEP (lateral movement, execution & persistence). Post-compromise actions that expand control after an initial foothold, including credential misuse and lateral authentication, execution of malicious payloads or commands, and persistence/privilege escalation behaviours that enable continued access and pivoting across internal components.
- IMP (impact). Actions that directly affect availability, integrity, or reliability of supply-chain assets or services, such as denial-of-service against gateways/servers, disruptive configuration changes, or manipulation of operational telemetry with immediate consequence.

Record-level labelling

In the reported experiments, each network-flow record is assigned exactly one label: Benign or one of {IAD, LMEP, IMP}. The wider framework is conceptually extensible to host and device telemetry, but these additional modalities are not part of the empirical evaluation.

Staging limitations and Interpretation

We do not claim perfect ATT&CK fidelity at the record level. Some behaviours (e.g., ransomware, backdoors, and credential abuse) can span multiple tactics depending on context and intent. Accordingly, each entity–window instance is assigned to the *dominant operational role* evidenced in the available telemetry (IAD vs. LMEP vs. IMP) rather than to an exhaustive set of tactics. The goal is operational abstraction for learning and triage, not fine-grained attribution.

Deterministic stage assignment rules

To ensure consistent labelling, we apply deterministic resolution rules when ambiguity arises. Specifically, (i) actions primarily intended to map the environment or test exposure are assigned to IAD; (ii) actions that establish or use access to traverse internal components or elevate privileges are assigned to LMEP; and (iii) actions whose immediate effect is disruption or manipulation of operations are assigned to IMP. For example, port scanning and service enumeration are mapped to IAD, credential misuse on internal services is mapped to LMEP, and denial-of-service activity is mapped to IMP. These rules prioritise campaign intent and observable operational effect, thereby reducing optimistic leakage of later-stage semantics into early-stage labels.

Rationale

This three-stage abstraction provides a compact, operationally interpretable view of campaign progression that can be learned from heterogeneous telemetry and supports downstream impact prediction by explicitly modelling progression toward high-severity outcomes.

Why a three-stage abstraction (granularity vs. robustness).

We deliberately adopt a coarse three-stage abstraction as an operational trade-off. Finer-grained ATT&CK tactics/techniques can increase interpretive detail, but in heterogeneous IIoT telemetry they often suffer from (i) label ambiguity (multiple tactics plausible for the same observable pattern), (ii) sparsity (rare techniques yield unstable learners and unreliable calibration), and (iii) forecasting instability (too many

fine states create high-variance transition dynamics and degrade pre-impact prediction under rare positives). Collapsing behaviours into IAD/LMEP/IMP preserves actionable semantics (early foothold, post-compromise expansion, operational impact) while improving robustness, class support, and stability for leakage-controlled forecasting.

Scope and ambiguity of the stage abstraction

The IAD/LMEP/IMP abstraction is intended as an operational learning representation rather than a replacement for the full MITRE ATT&CK for ICS taxonomy. Real attacks need not follow a strictly ordered IAD→LMEP→IMP progression. For example, persistence may be established without subsequent operational impact, lateral movement may occur without observable discovery, and behaviours such as credential abuse, backdoors, ransomware, injection, and man-in-the-middle activity may correspond to different tactics depending on attacker intent and system context. We therefore interpret each stage label as the *dominant operational role* represented by the available dataset observation, rather than as an exhaustive ground-truth ATT&CK annotation.

To quantify the dependence of the results on this modelling choice, Sect. 5.2.5 evaluates alternative conservative and expanded stage assignments for ambiguous attack labels under otherwise identical experimental settings.

4 Multi-stage detection and prediction framework

4.1 Overview

Figure 1 summarises the framework. The empirically evaluated Multi-Stage Detection Layer represents security observations using tabular features and communication relationships and produces stage-aware predictions over Benign, IAD, LMEP, and IMP. We compare a context-agnostic Random Forest, a context-aware Graph Convolutional Network, probability-level late fusion, and an out-of-fold stacking meta-model.

The framework additionally defines a Pre-Impact Forecasting Layer in which Stage 1/Stage 2 representations preceding an asset-specific cutoff would be processed by separate GRU encoders to estimate subsequent IMP risk. This component is retained as a methodological design. The revised real-data audit showed that neither evaluated benchmark representation provides sufficient valid longitudinal pre-impact trajectories for statistically meaningful forecasting evaluation; therefore, no empirical forecasting-performance claims are made.

Strict pre-impact forecasting is a core contribution

We treat leakage control as a first-class design constraint: for every asset, we define a cutoff time t_a and construct *all* predictor inputs using only observations with timestamps $< t_a$. This applies both to temporal sequences (Stage 1/2 streams) and to any graph-based embeddings (graphs restricted to pre-t_a nodes/edges). Under no condition is IMP-stage evidence used as an input feature; IMP is used only as supervision to define the prediction target.

4.2 Input telemetry and event set

Let $\mathcal{E} = \{e_i\}_{i=1}^{N}$ denote the set of security-relevant observations used for stage detection. In the reproducible experiments reported in this study, observations are derived from network telemetry and associated with a source/destination identity, available timing information, and attack label. The framework itself is extensible to additional host or device modalities, but such multi-modal fusion is not claimed as part of the validated experiments reported here. Following our staging definition, each event is assigned a label $y_i \in \mathcal{Y}$ where:

$$\mathcal{Y} = \{\text{Benign, IAD, LMEP, IMP}\}. \tag{1}$$

The detection layer learns a stage classifier that delivers an event-level probability vector:

$$\hat{\mathbf{p}}_i \in [0,1]^{|\mathcal{Y}|}, \qquad \sum_{c\in\mathcal{Y}} \hat{p}_{i,c} = 1, \tag{2}$$

and an event-level predicted stage $\hat{y}_i = \arg\max_{c\in\mathcal{Y}} \hat{p}_{i,c}$, which triggers stage alerts.

4.3 Feature vector & graph construction

As depicted in the upper part of Fig. 1, the framework first constructs (i) unified event feature vectors and (ii) an *asset–time window interaction graph*. This supports two complementary perspectives: a feature-only view (suitable for RF) and a relational-context view (suitable for GCN).

4.3.1 Unified feature construction

In the validated experiments, each network record is mapped to a common analysis representation and aggregated by asset and analysis window. The framework is conceptually extensible to additional host and IoT telemetry, but such multi-modal fusion is not part of the empirical evaluation reported in this study. Concretely, we define an asset–window instance e_i by an *entity–time window* tuple (e.g., host/IP or IoT device within a fixed window Δt), and construct a fixed-length feature vector $\mathbf{x}_i \in \mathbb{R}^F$ by aggregating all records belonging to that entity and window. This yields a consistent schema that supports heterogeneous sources while remaining compatible with both tabular and graph-based models.

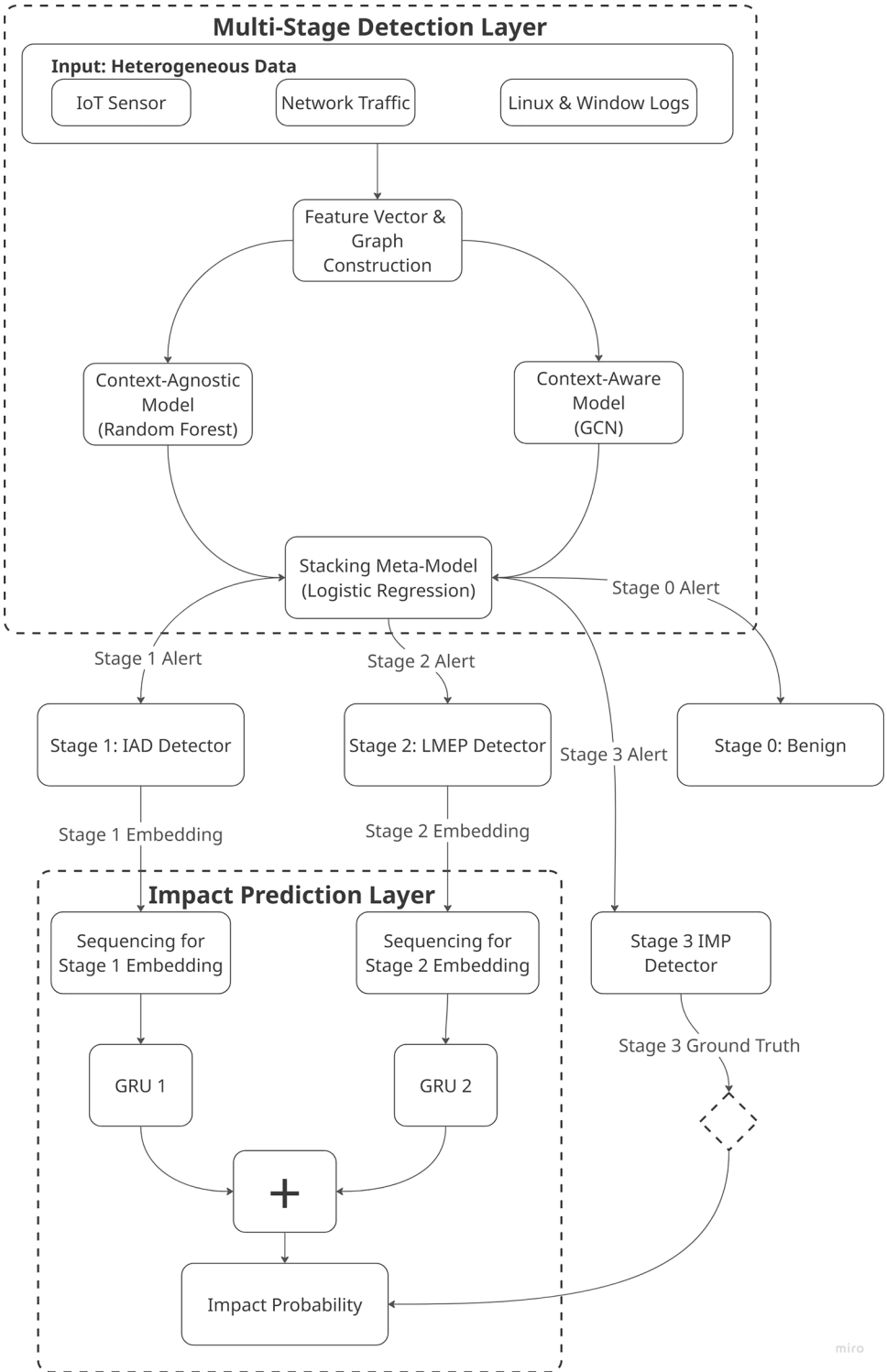


Fig. 1 Conceptual architecture of the stage-aware detection and pre-impact forecasting framework. The empirically evaluated detection layer represents observed network entities using feature vectors and communication-graph context and compares RF, GCN, late-fusion, and stacking strategies. The lower dual-stream GRU component denotes the proposed leakage-controlled forecasting extension, in which only pre-impact IAD/LMEP evidence would be admissible. The forecasting component is not assigned empirical performance claims in the revised study because the validated datasets do not provide sufficient longitudinal pre-impact trajectories

- Numerical features. Continuous traffic attributes such as packet counts, byte counts, durations, and related flow statistics are transformed where appropriate and standardised using statistics fitted exclusively on training data.
- *Categorical features.* Protocol, service, connection-state, and other categorical fields are encoded using transformations fitted only on the training split.
- *Identifier-aware de-biasing.* Direct identifiers that would allow trivial memorisation of a specific asset are excluded from the predictive feature vector. Asset identities are

retained separately only for splitting and graph construction.

- *Missing values and dataset-specific schema handling.* Missing or unsupported fields are handled through dataset adapters and deterministic preprocessing rules. All preprocessing objects are fitted on training data and then applied unchanged to validation and test data.

The resulting vector $\mathbf{x}_i$ is used directly as input to the context-agnostic classifier (Random Forest). The same per-entity vectors also define node features in the context-aware component, where entities form nodes and network interactions observed within each window induce edges (optionally weighted by interaction volume), enabling a GCN to exploit relational structure while sharing the same unified feature schema.

4.3.2 Graph definition and semantics

Entity (asset) definition. In the validated experiments, an entity $a \in \mathcal{A}$ denotes the network host identity used for identity-preserving splitting and graph construction. Extension to host-log or IoT-device identities follows the same conceptual framework but is not empirically evaluated in this study.

asset–window instance as an entity–time window aggregate. To unify heterogeneous records, we define a discrete windowing operator with window size Δt.

Let $m \in \{0, 1, \ldots\}$ index time windows and let the m-th window be $W_m = [m\Delta t, (m+1)\Delta t)$. For each entity a and window W_m, we construct a single *asset–window instance*

$$e_{a,m} \triangleq (a, W_m), \tag{3}$$

by aggregating all network records associated with a whose timestamps fall in W_m into a fixed-length feature vector $\mathbf{x}_{a,m} \in \mathbb{R}^F$.

Throughout the paper, we sometimes refer to $e_{a,m}$ as an "event" for brevity; it is more precisely an *entity–window aggregate*. Each node therefore corresponds to one entity observed over one fixed window.

Graph construction (train/test built independently). For a given split $s \in \{\text{train}, \text{meta-train}, \text{test}\}$, we build a graph $G^{(s)} = (V^{(s)}, E^{(s)})$ using *only* the entities and telemetry belonging to that split. This yields separate graphs for training/meta-training/testing and prevents any cross-split edges or neighborhood leakage. Nodes are

$$V^{(s)} = \{v_{a,m} \mid e_{a,m} \text{ exists in split } s\}, \tag{4}$$

$$\mathbf{X}^{(s)} = [\mathbf{x}_{a,m}]_{v_{a,m} \in V^{(s)}} \in \mathbb{R}^{|V^{(s)}| \times F}. \tag{5}$$

We rebuild the graph separately for base-train, meta-train, and test splits (no cross-split edges) to prevent identity/neighbourhood leakage; graphs are cached and reused across seeds within a fixed split and window size.

Edges: interaction and temporal continuity. We add two edge sets and then take their union:

$$E^{(s)} = E^{(s)}_{\text{com}} \cup E^{(s)}_{\text{tmp}}. \tag{6}$$

(i) Interaction (communication) edges. Within each window W_m, if the network telemetry indicates that entity a communicates with entity b during W_m, we add an interaction edge between the corresponding window-nodes:

$$(v_{a,m}, v_{b,m}) \in E^{(s)}_{\text{com}} \iff \exists \text{ a flow/event linking } a \leftrightarrow b \text{ with timestamp in } W_m. \tag{7}$$

Because our GCN uses a standard (untyped) message-passing operator, we treat these edges as *undirected* by symmetrising the adjacency. When multiple connections occur in the same window, we optionally define a nonnegative edge weight w^{com}_{abm} proportional to interaction volume (e.g., flow count or total bytes), and apply $\log(1+w)$ scaling to reduce heavy-tailed effects.

(ii) Temporal continuity edges. For each entity a, we connect consecutive windows to preserve within-asset progression:

$$(v_{a,m}, v_{a,m+1}) \in E^{(s)}_{\text{tmp}} \iff e_{a,m} \text{ and } e_{a,m+1} \text{ exist}, \tag{8}$$

with unit weight $w^{\text{tmp}} = 1$ (or a constant) and treated as undirected for the GCN. These edges ensure that entities with sparse interaction evidence (e.g., IoT-only windows) can still contribute temporal structure.

Edge typing, self-loops, and normalisation. The construction yields two semantic edge sets, E_{com} and E_{tmp}. In the main configuration, we collapse them into a single untyped adjacency and use a standard GCN. This provides a simple and computationally tractable reference architecture while retaining both communication and temporal-continuity information. Self-loops are added to all nodes and standard symmetric normalisation $\tilde{\mathbf{D}}^{-1/2}\tilde{\mathbf{A}}\tilde{\mathbf{D}}^{-1/2}$ is applied. When interaction-edge weights are used, $\tilde{\mathbf{A}}$ is weighted after symmetrisation and self-loop insertion. We do not claim that the homogeneous undirected representation is optimal; interaction-only, temporal-only, and combined-edge variants are compared experimentally, while relational, directed, and heterogeneous graph architectures remain future work.

Edge-typed variants (interaction vs. temporal). We considered edge-typed alternatives (e.g., relational GCN or

type-specific message passing) that distinguish interaction vs. temporal continuity. In our setting, the primary benefit of typing is interpretability, while the cost is higher memory and slower training. We therefore report the untyped GCN as the main model and include an ablation that isolates the contribution of each edge set (interaction-only vs. temporal-only vs. both). If edge-typed models are evaluated, we report their incremental gains (e.g., ΔMacro-F1) alongside resource overhead to justify the design trade-off.

Typical node density and modality-driven sparsity. Because nodes are formed by entity–window aggregation, the number of nodes per asset depends on how long (and how frequently) the asset emits records. Interaction edges are induced primarily by network telemetry; thus, entities with limited network visibility (e.g., some IoT streams) may have fewer interaction edges, but still remain connected via temporal edges.

Graph scale. Graph scale is dataset dependent and is reported directly in Tables 3 and 4. Following correction of the ToN-IoT temporal-windowing pipeline, the ToN-IoT graph contains 34,435 asset–window nodes, 10,168 interaction edges, and 22,899 temporal-continuity edges, giving 33,067 edges in total and an approximate average degree of 1.92. The presence of temporal edges confirms that the corrected representation contains genuine multi-window asset trajectories. Edge-IIoTset contains 142,095 record-level graph nodes and 17,685 interaction edges; temporal edges are not constructed because the published ML representation does not preserve reliable longitudinal timestamps.

4.4 Multi-stage detection layer

The unified per-entity–window vectors and the corresponding windowed interaction graph are processed by two complementary base learners: a context-agnostic tabular classifier (Random Forest) and a context-aware graph classifier (GCN). Their probabilistic outputs are then fused via stacking to produce final stage alerts consistent with Fig. 1.

4.4.1 Context-agnostic stage classifier (random forest)

The context-agnostic branch operates purely on the unified feature vector $\mathbf{x}_i$ for asset–window instance e_i (an entity–window aggregate). A Random Forest (RF) is trained to predict stage probabilities over the stage label set $\mathcal{Y}$:

$$\hat{\mathbf{p}}_i^{\mathrm{RF}} = f_{\mathrm{RF}}(\mathbf{x}_i), \qquad \hat{\mathbf{p}}_i^{\mathrm{RF}} \in [0,1]^{|\mathcal{Y}|}. \tag{9}$$

RF is well-suited to tabular traffic features and captures non-linear interactions among protocol, connection, and flow-level attributes while allowing each entity–window to be evaluated independently. It provides a strong and efficient scorer for high-throughput environments, where each entity–window can be evaluated independently.

4.4.2 Context-aware stage classifier (GCN)

The context-aware branch leverages the graph $G = (V, E)$ constructed in Sect. 4.3.2, where each node corresponds to an entity–window event and edges encode (i) communication/interaction relations derived from network flows and (ii) temporal continuity across consecutive windows for the same entity. Let $\tilde{\mathbf{A}}$ denote the (optionally weighted) adjacency matrix with added self-loops and $\tilde{\mathbf{D}}$ its degree matrix. A standard GCN layer performs neighbourhood aggregation:

$$\mathbf{H}^{(l+1)} = \sigma\left(\tilde{\mathbf{D}}^{-\frac{1}{2}}\tilde{\mathbf{A}}\tilde{\mathbf{D}}^{-\frac{1}{2}}\mathbf{H}^{(l)}\mathbf{W}^{(l)}\right), \tag{10}$$

with $\mathbf{H}^{(0)} = \mathbf{X}$, where $\mathbf{X} = [\mathbf{x}_1; \ldots; \mathbf{x}_N]$ is the node feature matrix. The final layer yields stage probabilities:

$$\hat{\mathbf{p}}_i^{\mathrm{GCN}} = \mathrm{softmax}\left(\mathbf{h}_i^{(L)}\mathbf{W}^{(L)}\right), \qquad \hat{\mathbf{p}}_i^{\mathrm{GCN}} \in [0,1]^{|\mathcal{Y}|}. \tag{11}$$

Intuitively, the GCN disambiguates stages whose signatures are primarily relational or sequential, e.g., discovery-like behaviour emerging across many related connections, coordinated probing against shared ports/services, or progression patterns where subtle host/IoT deviations become meaningful only when viewed alongside temporally adjacent activity.

4.4.3 Stacking meta-model (logistic regression) and alerts

As shown in Fig. 1, we fuse RF and GCN outputs via stacking. For each event e_i, we concatenate the two probability vectors:

$$\mathbf{s}_i = \left[\hat{\mathbf{p}}_i^{\mathrm{RF}} \,\|\, \hat{\mathbf{p}}_i^{\mathrm{GCN}}\right] \in \mathbb{R}^{2|\mathcal{Y}|}. \tag{12}$$

A logistic regression meta-model $g(\cdot)$ maps $\mathbf{s}_i$ to final stage probabilities:

$$\hat{\mathbf{p}}_i = g(\mathbf{s}_i), \qquad \hat{y}_i = \arg\max_{c \in \mathcal{Y}} \hat{p}_{i,c}. \tag{13}$$

The predicted stage $\hat{y}_i$ triggers an event-level stage alert. Following Fig. 1, Stage 1 alerts correspond to IAD detections, Stage 2 alerts to LMEP detections, and Stage 3 alerts to IMP detections.

Asset-disjoint out-of-fold stacking. To prevent optimistic bias and to preserve identity-based generalisation, we employ an asset-disjoint out-of-fold protocol. Training assets are split into a *base-training* subset and a disjoint *meta-training* subset (no shared host/device identities). The RF and GCN

Algorithm 1 Asset-disjoint stacking protocol (RF–GCN → logistic meta-learner)

Require: Training assets $\mathcal{A}_{train}$, disjoint test assets $\mathcal{A}_{test}$; labels y; features $\mathbf{X}$; graph builder $\mathsf{BuildGraph}(\cdot)$.

1: Split $\mathcal{A}_{train} = \mathcal{A}_{base} \cup \mathcal{A}_{meta}$ with $\mathcal{A}_{base} \cap \mathcal{A}_{meta} = \emptyset$
2: Train RF on instances from $\mathcal{A}_{base}$: $f_{\mathrm{RF}} \leftarrow \mathsf{TrainRF}(\mathbf{X}_{base}, y_{base})$
3: Build graph on $\mathcal{A}_{base}$ only and train GCN: $G_{base} \leftarrow \mathsf{BuildGraph}(\mathcal{A}_{base})$; $f_{\mathrm{GCN}} \leftarrow \mathsf{TrainGCN}(G_{base})$
4: Generate *out-of-sample* meta features on $\mathcal{A}_{meta}$: $\mathbf{s}_i = [\hat{\mathbf{p}}_i^{RF} \| \hat{\mathbf{p}}_i^{GCN}]$ for all $i \in \mathcal{A}_{meta}$
5: Fit logistic meta-learner on meta instances: $g \leftarrow \mathsf{TrainLR}(\mathbf{s}_{meta}, y_{meta})$
6: Retrain base models on all training assets $\mathcal{A}_{train}$: $f_{\mathrm{RF}} \leftarrow \mathsf{TrainRF}(\mathbf{X}_{train}, y_{train})$; $G_{train} \leftarrow \mathsf{BuildGraph}(\mathcal{A}_{train})$; $f_{\mathrm{GCN}} \leftarrow \mathsf{TrainGCN}(G_{train})$
7: For test assets $\mathcal{A}_{test}$, compute $\mathbf{s}_i$ and output $\hat{\mathbf{p}}_i = g(\mathbf{s}_i)$

are trained on the base-training subset and generate out-of-sample probability vectors $\mathbf{s}_i$ for events in the meta-training subset. The logistic regression meta-model is then fitted on these meta-training predictions. At test time, the RF and GCN are retrained on the full training assets (still disjoint from test assets), and their outputs are fused by the fixed meta-model to produce final stage probabilities.

4.4.4 Stage-specific embeddings for impact prediction

Beyond generating alerts, the detection layer produces stage-specific representations that serve as inputs to the impact prediction layer. For events predicted as Stage 1 (IAD) or Stage 2 (LMEP), we extract embeddings that summarise both the unified features and (when applicable) neighbourhood context:

$$\mathbf{z}_i^{(1)} = \psi_1(e_i), \qquad \mathbf{z}_i^{(2)} = \psi_2(e_i). \tag{14}$$

In practice, $\psi_1(\cdot)$ and $\psi_2(\cdot)$ are instantiated as either: (i) the final GCN node embedding $\mathbf{h}_i^{(L)}$ (capturing relational and temporal context), or (ii) the fused probability vector $\hat{\mathbf{p}}_i$ (capturing stage confidence and uncertainty). Importantly, consistent with Fig. 1, the impact predictor consumes only Stage 1/Stage 2 embeddings (early-stage evidence) rather than relying on Stage 3 labels/evidence, enabling forward-looking impact estimation before final-stage confirmation.

4.5 Impact prediction layer

This layer provides a proactive *impact-risk forecast*: for each asset (host or IoT device), it predicts the likelihood of escalation to *Stage 3* (IMP) in future, using only evidence observed before a cutoff time t. Rather than reacting to confirmed impact, it converts *early-stage signals* from Stage 1 (IAD) and Stage 2 (LMEP) into an actionable early-warning score, explicitly excluding any Stage 3 observations to ensure the prediction reflects genuine pre-impact risk.

4.5.1 Sequence construction (Stage 1 and Stage 2 streams)

For every asset, we build two separate time-ordered histories from events that occur strictly before the cutoff time t:

- A *Stage 1 stream* containing embeddings derived from events predicted as Stage 1 (IAD).
- A *Stage 2 stream* containing embeddings derived from events predicted as Stage 2 (LMEP).

Each stream is represented as a fixed-length sequence of the most recent T embeddings before t. If an asset has fewer than T events in a stream, we pad the sequence and use a mask so that padding does not affect learning. This ensures that impact prediction is driven by early-stage patterns rather than late-stage evidence.

4.5.2 Dual GRU encoders

Why two streams instead of early fusion. Stage 1 (IAD) and Stage 2 (LMEP) exhibit different temporal signatures: IAD is often bursty and exploratory (scans, probing, credential attempts), whereas LMEP tends to be more stateful and persistent (execution, repeated authentication, privilege/persistence signals). Early fusion of Stage 1 and Stage 2 into a single sequence can dilute these distinct dynamics and encourages the model to learn an "average" pattern dominated by the more frequent stage (often LMEP), which reduces sensitivity to weak but early IAD cues. The dual-stream design preserves stage-specific dynamics and allows the predictor to weight them differently when estimating escalation risk.

We use two GRU encoders, one per stream, to model how Stage 1 and Stage 2 activity evolves over time. Keeping the streams separate allows the model to learn different temporal signatures for early reconnaissance/foothold behaviours (Stage 1) versus movement and escalation behaviours (Stage 2). This is important because Stage 1 and Stage 2 often differ in their frequency, burstiness, and persistence, especially across mixed telemetry sources (network flows, host metrics, and IoT sensors).

4.5.3 Fusion and impact probability

After encoding the two streams, we combine their representations using element-wise addition. The fused representation is passed through a small prediction head to output a single value between 0 and 1. This value is interpreted as the risk score that the asset will reach Stage 3 in the future.

4.5.4 Stage 3 ground truth and leakage control

This layer provides a proactive *impact-risk forecast*: for each asset (host or IoT device), it predicts the likelihood of escalation to *Stage 3* (IMP) in future, using only evidence observed before a cutoff time t.

This setup enforces the intended proactive setting: Stage 3 is never used as an input signal, but only as the label that we aim to predict in advance. We train the model using a standard binary classification loss, and apply class weighting (or focal loss) if needed to handle the natural imbalance between impacted and non-impacted assets.

Preventing temporal leakage. To ensure the predictor does not "peek into the future", we strictly enforce that all inputs are built only from events occurring before the cutoff time t. In particular:

- The Stage 1 and Stage 2 embeddings used to form the two sequences are computed only from records prior to t.
- When graph-based embeddings are used, the graph is restricted to nodes and edges derived only from pre-t telemetry (i.e., excluding flows and host/IoT observations at or after t).

This guarantees that the impact score reflects genuinely early-stage evidence rather than information that becomes available after Stage 3 occurs.

5 Experiments

5.1 Dataset description

5.1.1 Data sources and modalities

We evaluate stage-aware intrusion detection using two public IoT/IIoT cybersecurity benchmarks: ToN-IoT and Edge-IIoTset. For ToN-IoT, the reproducible experiments in this study use the labelled network-flow dataset, which provides Unix-epoch timestamps, source and destination host identifiers, protocol and connection features, and attack labels suitable for identity-preserving stage classification and graph construction. The dataset contains benign traffic together with scanning, DoS/DDoS, injection, password, cross-site scripting, ransomware, backdoor, and man-in-the-middle activity.

To assess cross-dataset portability, we additionally use Edge-IIoTset, which contains labelled IoT/IIoT network observations spanning multiple benign and malicious behaviours. Using two independently constructed benchmarks enables us to assess whether the proposed IAD/LMEP/IMP operational abstraction and stage-detection methodology are specific to a single dataset or remain applicable under a different traffic and attack distribution.

5.1.2 External validation dataset: edge-IIoTset

To address the limitation of single-dataset evaluation, we include Edge-IIoTset as an independent external IoT/IIoT benchmark. Edge-IIoTset contains labelled benign and malicious network observations collected from an IoT/IIoT cybersecurity testbed and includes attack categories such as fingerprinting, vulnerability scanning, password attacks, backdoor activity, injection, ransomware, man-in-the-middle activity, and multiple denial-of-service variants.

In this work, Edge-IIoTset is used to evaluate cross-dataset portability of the stage-aware detection task. Its attack labels are mapped into the same Benign/IAD/LMEP/IMP operational abstraction used for ToN-IoT. During dataset validation, we found that the selected `ML-EdgeIIoT-dataset.csv` file does not retain complete calendar information in the `frame.time` field for reliable longitudinal reconstruction; a subset of records also contains placeholder timestamp values. We therefore do not use Edge-IIoTset for temporal-window sensitivity, temporal continuity modelling, or pre-impact forecasting. This restriction prevents automatic date parsing from introducing artificial chronology and ensures that the external-validation results are limited to tasks directly supported by the published data (Table 2).

5.1.3 Dataset instances and identity-preserving evaluation

For ToN-IoT, the revised experiments use the real `Train_Test_Network.csv` dataset containing 461,043 labelled network observations with Unix-epoch timestamps, source and destination host information, and attack labels. For Edge-IIoTset, we use the published `ML-EdgeIIoT-dataset.csv` containing 157,800 labelled observations. Dataset-specific adapters map the original schemas into the common stage-detection representation. Where stable asset identities are available, splitting is performed by identity rather than by randomly assigning individual observations. All observations associated with a given identity are therefore restricted to one evaluation partition, reducing the risk that asset-specific behaviour is memorised across train and test sets. Tables 3 and 4 report the final real-data characteristics used by the models (Tables 3, 4).

5.1.4 Preprocessing and unified representation

In the validated experiments, preprocessing operates on network-flow telemetry. Numerical traffic features are transformed where appropriate and standardised using statistics computed exclusively from the training partition. Categorical

Table 2 Role of the two datasets in the revised experimental evaluation

Dataset	Primary role	Evaluation scope
ToN-IoT	Primary real-data benchmark	Stage-aware classification, interaction-graph evaluation, graph ablations, stage-mapping sensitivity, baseline comparison, bootstrap uncertainty analysis, and inference-latency evaluation
Edge-IIoTset	Independent external benchmark	Cross-dataset stage-aware classification, static interaction-graph evaluation where supported, stage-mapping sensitivity, baseline comparison, bootstrap uncertainty analysis, and inference-latency evaluation

Table 3 Summary of the corrected real-data ToN-IoT representation used in the experiments

Characteristic	Value
Raw data source	Network-flow telemetry
Raw records	461,043
Unit of analysis	Asset–time window
Window size (Δt)	60 s
Final asset–window instances	34,435
Distinct assets	11,536
Feature dimension	126
Stage distribution	
Benign	33,808
IAD	86
LMEP	298
IMP	243
Data partitions	
Training instances	27,326
Validation instances	3,796
Test instances	3,313
Graph characteristics	
Graph nodes	34,435
Interaction edges	10,168
Temporal edges	22,899
Total edges	33,067
Average degree	≈ 1.92
Pre-impact forecasting instances	21,313
Positive forecasts (Train / Val / Test)	5 / 0 / 0
Forecasting evaluation	Not statistically valid[a]

[a]Correct temporal reconstruction yields genuine longitudinal asset trajectories and 21,313 candidate forecasting instances. However, only five instances are positive and all occur in the training partition; validation and test contain no positive forecasting cases. Consequently, statistically meaningful held-out forecasting metrics cannot be estimated

Table 4 Summary of the final real-data Edge-IIoTset representation used in the experiments

Characteristic	Value
Raw data source	Network-flow telemetry
Raw records	157,800
Records retained after preprocessing	142,095
Unit of analysis	Record level[a]
Final instances	142,095
Feature dimension	175
Distinct assets	17,687
Stage distribution	
Benign	24,301
IAD	41,189
LMEP	30,775
IMP	45,830
Data partitions	
Training	68,301
Validation	10,804
Test	62,990
Graph characteristics	
Graph nodes	142,095
Interaction edges	17,685
Temporal edges	0
Average degree	0.249
Temporal forecasting evaluation	Not valid[a]

[a]The published `ML-EdgeIIoT-dataset.csv` does not preserve complete calendar dates in `frame.time`; therefore, reliable temporal windowing, temporal-continuity edges, and leakage-controlled pre-impact forecasting cannot be constructed from this representation

traffic attributes are encoded using transformations fitted on training data only. Direct identifiers and label-derived artefacts that could enable trivial memorisation are excluded from the predictive feature vector, while asset identities are retained separately for identity-preserving splitting and graph construction. Missing or unsupported fields are handled deterministically by the dataset-specific adapters.

5.1.5 Edge-IIoTset preprocessing

For Edge-IIoTset, we follow the same leakage-aware preprocessing principles used for ToN-IoT. Numerical features are

standardised using statistics computed only on the training split, categorical variables are encoded using training-fitted encoders, and non-generalising identifiers or label-derived artefacts are removed where applicable. The original attack labels are first mapped to Benign/IAD/LMEP/IMP according to Table 8. We then evaluate four-class stage classification using the same macro-F1-oriented protocol. Where reliable asset identifiers and timestamps are available, we additionally construct entity–time windows and graph edges using the same procedure as ToN-IoT. Where such identifiers are unavailable or insufficient for leakage-controlled graph construction, Edge-IIoTset is used for feature-based and stage-mapping validation only, and this limitation is explicitly reported.

5.1.6 Final real-data characteristics

Tables 3 and 4 report the characteristics used in the final experiments. All statistics are generated from the validated real-data pipeline. ToN-IoT yields entity–window instances with reliable Unix timestamps, whereas Edge-IIoTset is evaluated at record level because its published ML representation does not preserve complete calendar information required for reliable temporal reconstruction.

5.2 Stage label mapping

To align ToN-IoT attack-type labels with our ATT&CK-inspired staging (Stage 1: IAD, Stage 2: LMEP, Stage 3: IMP), we map each ToN-IoT attack type to one or more candidate MITRE ATT&CK for ICS techniques using publicly documented technique definitions and the ToN-IoT attack descriptions. The goal of this mapping is *operational abstraction*: we do not claim a one-to-one equivalence between dataset labels and ATT&CK techniques, but rather a principled grouping that reflects the *primary campaign role* of each attack type and supports interpretable stage-aware learning.

5.2.1 Mapping procedure

We performed mapping in three steps: (i) *Candidate technique selection:* for each ToN-IoT attack type, we selected a small set of plausible ATT&CK for ICS techniques based on the technique definition and expected observable behaviour (e.g., discovery, credential abuse, remote access, manipulation, disruption). (ii) *Primary-role assignment:* when multiple techniques were plausible, we assigned the attack type to the technique that best matches its *dominant operational intent and immediate effect* in ToN-IoT (e.g., disruption → IMP). Secondary candidates were retained for traceability (reported in Table 5). (iii) *Stage aggregation:* we then aggregated mapped techniques into stages: discovery/foothold behaviours → IAD; post-compromise expansion and persistence behaviours → LMEP; direct service/process disruption → IMP.

5.2.2 Mapping verification and reproducibility

The mapping is defined by the deterministic stage-assignment rubric in Sect. 3 and cross-checked against MITRE ATT&CK for ICS technique definitions. Four researchers independently reviewed the final attack-type→stage mapping for rubric consistency (IAD vs. LMEP vs. IMP). We report inter-annotator agreement as percentage consensus and, where applicable, Cohen's/Fleiss' κ to support reproducibility: consensus $= 83.3\%$ and $\kappa = 0.78$. Disagreements were resolved by applying the rubric verbatim. We emphasise that this mapping is an *operational abstraction* over dataset attack types; it is not a ground-truth ATT&CK annotation of the testbed.

5.2.3 Final mapping used in experiments

Using the validated mapping (`configs/stage_mapping_toniot.yaml`), the primary stage aggregation is:

IAD: Scanning, Password, XSS;
LMEP: Backdoor, Injection, MITM; and
IMP: DoS, DDoS, Ransomware.

Where an attack type could plausibly span multiple ATT&CK tactics, we prioritise the dominant operational role and immediate observable effect, consistent with the staging rubric in Sect. 3. Alternative conservative and expanded mappings are evaluated in the stage-mapping sensitivity analysis (Fig. 2, Table5).

5.2.4 Edge-IIoTset stage mapping

For Edge-IIoTset, we apply the same dominant-role mapping principle used for ToN-IoT. Attack types primarily associated with reconnaissance, probing, or initial access attempts are mapped to IAD. Attack types involving exploitation, credential abuse, malicious command execution, injection, backdoor behaviour, or traffic manipulation are mapped to LMEP. Attack types whose immediate observable effect is service degradation, denial of service, or operational disruption are mapped to IMP. Benign traffic remains a separate class.

5.2.5 Stage-mapping sensitivity analysis

We evaluate three stage-assignment configurations: (i) the primary mapping based on dominant operational role, (ii) a conservative-impact mapping in which ambiguous manipulation/execution behaviours remain in LMEP unless direct disruption is observable, and (iii) an expanded-impact mapping in which selected manipulation-oriented behaviours are

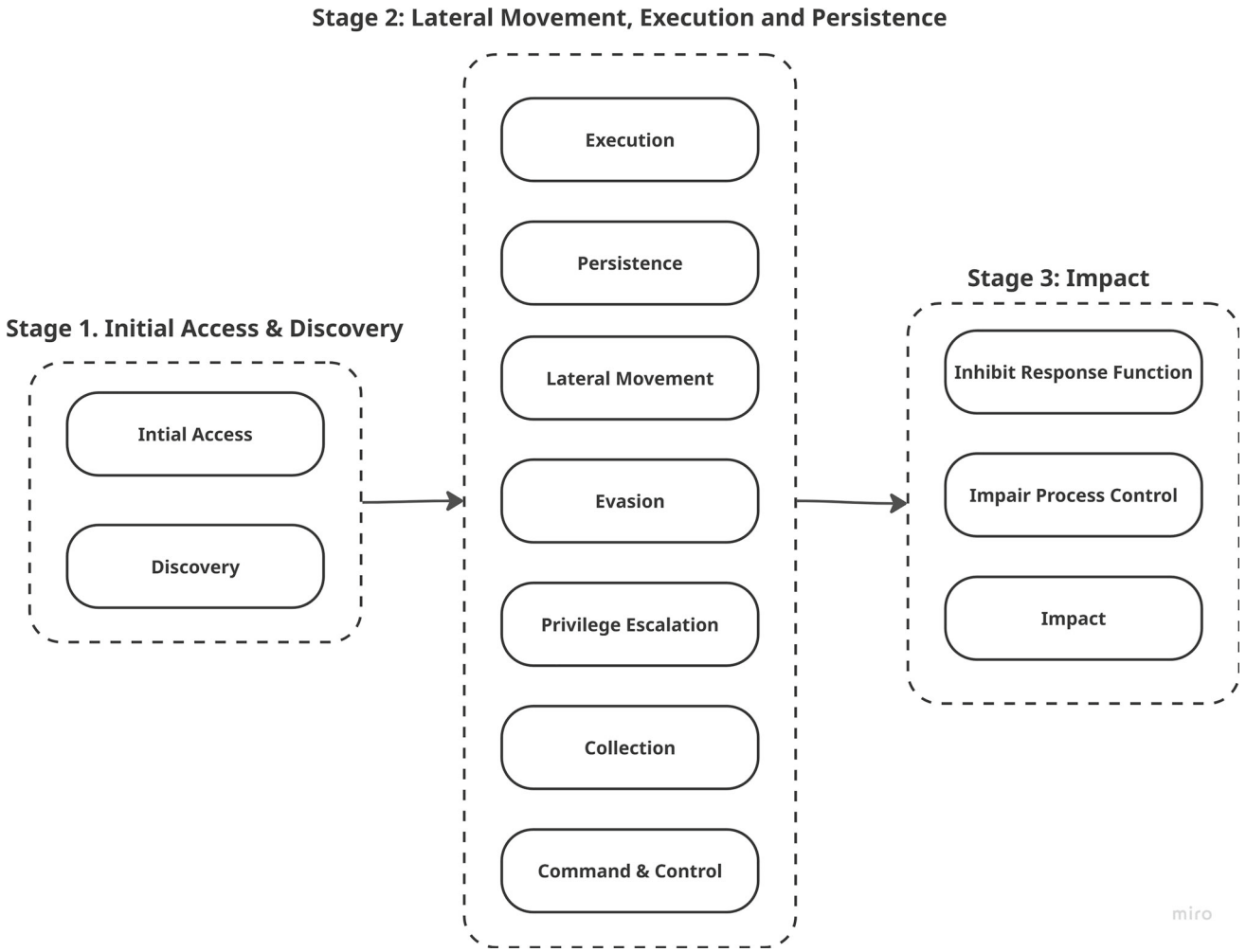


Fig. 2 Semantics-based mapping from ToN-IoT attack types to MITRE ATT&CK for ICS techniques and aggregation into the three stages used in this work: Stage 1 (IAD), Stage 2 (LMEP), and Stage 3 (IMP)

treated as IMP. The models are retrained under each mapping using otherwise identical experimental settings. Tables 6 and 7 report the resulting Macro-F1 and stage-specific performance. These experiments make the stage-definition assumption explicit and allow the robustness of the main conclusions to be assessed without treating one manually defined mapping as ground truth (Tables 6, 7, 8).

Where Edge-IIoTset labels do not exactly match the generic labels shown in Table 8, we apply the same rubric to the dataset-specific label names and report the final mapping in the released configuration file.

5.3 Tasks and evaluation protocol

5.3.1 Task 1: stage-wise detection (event-level)

Problem definition. Stage-wise detection is formulated as four-class event classification with labels {Benign, IAD, LMEP, IMP}. Each event corresponds to one unified instance

Table 5 Semantics-based mapping from ToN-IoT attack types to candidate MITRE ATT&CK for ICS technique(s)

ToN-IoT Attack Type	Mapped MITRE ATT&CK for ICS Technique(s)
Scanning	T0846 (Remote System Discovery)
Password	T0812 (Brute Force I/O), T0859 (Valid Accounts)
XSS	T0853 (Scripting)
Backdoor	T0859 (Valid Accounts), T0886 (Remote Services)
Injection	T0831 (Manipulation of Control), T0836 (Modify Parameter)
MITM	T0831 (Manipulation of Control), T0855 (Unauthorized Command Message), T0869 (Standard Application Layer Protocol)
DoS / DDoS	T0814 (Denial of Service)
Ransomware	T0826 (Loss of Availability)

Table 6 ToN-IoT stage-mapping sensitivity for the Stacked RF–GCN (seed 42), corrected after temporal-window reconstruction

Mapping	Macro-F1
Primary	0.308
Conservative	0.338
Expanded	0.414

after normalisation. We report event-level performance on held-out test identities.

5.3.2 Proposed Task 2: leakage-controlled impact forecasting

The following task defines the intended forecasting protocol but is not included in the final empirical performance comparison. During real-data validation, the available asset trajectories were found to be insufficient for constructing a statistically meaningful number of leakage-safe multi-window pre-impact examples. We retain the formulation to make the temporal and leakage requirements explicit and to support future evaluation on suitable longitudinal datasets.

Candidate forecasting instances. For an asset a with an ordered sequence of observation windows, we construct forecasting instances at eligible cutoff times $t_{a,j}$. Each forecasting instance contains only Stage 1/Stage 2 evidence observed strictly before its cutoff and is assigned a binary label indicating whether IMP occurs within the future interval $(t_{a,j}, t_{a,j} + H]$. Consequently, an asset may contribute multiple candidate forecasting instances, while all instances belonging to that asset remain in the same train, validation, or test partition under the asset-disjoint protocol.

Cutoff and label construction. A candidate cutoff is retained only when the required pre-cutoff history and subsequent observation interval can be constructed without using post-cutoff information. For each retained cutoff,

$$z_{a,j} = \begin{cases} 1, & \text{if an IMP event occurs in } (t_{a,j}, t_{a,j} + H], \\ 0, & \text{otherwise.} \end{cases} \tag{15}$$

Assets whose first observed window is already IMP do not provide a valid positive pre-impact forecasting trajectory because no earlier IAD/LMEP history is available.

Asset-level labels. The binary target for each asset is

$$z_a = \begin{cases} 1, & \text{if at least one IMP event occurs for asset } a \text{ in } (t_a, t_a + H], \\ 0, & \text{otherwise.} \end{cases} \tag{16}$$

Thus, z_a captures whether the asset escalates to operational impact within horizon H after the cutoff.

Leakage control for pre-impact inputs. All predictor inputs for asset a are constructed exclusively from events with timestamps $< t_a$. Stage 1 and Stage 2 sequences are formed from the most recent T embeddings (or probabilities) before t_a, using padding and masking when fewer than T events exist. When using graph-based embeddings, we compute embeddings from a graph restricted to nodes/edges derived only from telemetry with timestamps $< t_a$, so that no post-cutoff (including IMP) information can influence the representation.

5.3.3 Identity-preserving evaluation protocol

Asset-disjoint split. To evaluate generalisation to previously unseen assets, we adopt an *asset-disjoint* split: all events associated with a given asset identity (e.g., device identifier, host identifier, or internal IP) are assigned exclusively to either training/validation or test. This prevents *identity leakage* that can occur under record-level random splits, where the model may implicitly memorise asset-specific baselines and thereby overestimate performance. Asset-disjoint evaluation therefore provides a more realistic deployment proxy in IIoT environments, where detectors must operate on new devices and hosts not observed during training.

5.4 Graph construction details

We construct an *asset–time window interaction graph* in which each node corresponds to an *entity–time window* instance (Sect. 4.3.1). For a fixed window size Δt, we aggregate all raw records belonging to an entity (host/IP or IoT device identifier) within a window into a single node feature vector $\mathbf{x}_i$. Two edge types are added: (i) *interaction edges* linking nodes whose underlying entities communicate within the same window (from network flow tuples), and (ii) *temporal continuity edges* linking consecutive windows

Table 7 edgeiiotset: stage-mapping sensitivity

Mapping	Macro-F1	F1(IAD)	F1(LMEP)	F1(IMP)	PR-AUC	Capture@5%
primary	0.082	0.020	0.003	0.305	–	–
conservative	0.067	0.047	0.001	0.220	–	–
expanded	0.082	0.019	0.003	0.304	–	–

Table 8 Operational mapping from Edge-IIoTset attack labels to the proposed stage abstraction. Ambiguous labels are included in the sensitivity analysis

Edge-IIoTset attack label	Mapped stage	Rationale
Port scanning / vulnerability scanning / reconnaissance	IAD	Primarily maps exposure, services, or reachable devices before deeper compromise
Password / brute-force / credential attack	IAD or LMEP	Initial credential acquisition can be IAD; repeated internal credential misuse can be LMEP; handled in sensitivity analysis
Backdoor / malicious control / command execution	LMEP	Indicates post-compromise control, execution, or persistence behaviour
SQL injection / command injection / uploading attack	LMEP	Represents exploitation or manipulation after identifying an attack surface
MITM / ARP spoofing / traffic manipulation	LMEP	Enables interception, manipulation, or control of communication paths
DoS / DDoS	IMP	Directly affects service availability and operational continuity
Ransomware	IMP or LMEP	Can represent direct operational disruption or pre-impact execution; handled in sensitivity analysis
Data exfiltration / information theft, if present	LMEP / separate objective	Does not always correspond to availability/integrity impact; discussed as an alternative attacker objective
Benign / Normal	Benign	Non-attack class

of the same entity. In the main experiments, we use a fixed analysis-window duration of $\Delta t = 60$ s. This value is treated as a design configuration rather than as a validation-selected hyperparameter. We additionally report a post-hoc sensitivity analysis over alternative window sizes to examine how the Stacked RF–GCN behaves under different temporal aggregation scales. For reproducibility, we report graph statistics (nodes/edges and average degree) for the training and test graphs.

5.5 Graph-construction sensitivity

To examine sensitivity to temporal aggregation, we perform a descriptive post-hoc sweep using $\Delta t \in \{30, 60, 120, 300\}$ s. The sweep evaluates the Stacked RF–GCN at seed 42 and reports held-out test Macro-F1; it is therefore a sensitivity analysis rather than a validation-based hyperparameter-selection procedure.

The relationship between window size and performance is non-monotonic. Within this single-seed experiment, 120 s yields the highest observed Macro-F1 (0.579). Because the sweep is single-seed, model-specific, and evaluated descriptively on the held-out partition, we do not interpret 120 s as a globally optimal window size. The main experiments retain $\Delta t = 60$ s as the fixed design configuration (Tables 9, 11, 11).

Interaction edges alone outperform the combined graph in this controlled ablation (0.359 vs. 0.308 Macro-F1), indicating that simply combining interaction and temporal edges is not necessarily additive. The temporal edges nevertheless encode genuine chronological continuity and their presence confirms that the corrected windowing pipeline preserves multi-window asset behaviour. More expressive edge-type weighting or relational message passing is left for future work (Table 10).

Table 9 Post-hoc window-size sensitivity of the Stacked RF–GCN on ToN-IoT (seed 42). The sweep was not used for validation-based hyperparameter selection

Δt (s)	Graph nodes	Macro-F1	F1(IMP)
30	44,688	0.376	0.364
60	34,435	0.308	0.195
120	26,561	**0.579**	0.609
300	20,094	0.532	**0.727**

Bold values indicate the highest performance for each evaluation metric

Table 10 ToN-IoT graph edge-type ablation (seed 42), corrected after temporal-window reconstruction

Graph mode	Edges	Macro-F1	F1(IMP)
Interaction-only	10,168	**0.359**	**0.300**
Temporal-only	22,899	0.283	0.104
Both (main)	33,067	0.308	0.195

Bold values indicate the highest performance for each evaluation metric

Table 11 EdgeIIoTset graph ablation

Metric	Interaction-only	Self-loop-only	Both
Nodes	142,095	142,095	142,095
Edges	17,685	0	17,685
Average degree	0.249	0.000	0.249
Macro F1	0.115	0.228	0.115
F1 (Benign)	0.000	0.048	0.000
F1 (IAD)	0.085	0.404	0.085
F1 (LMEP)	0.000	0.000	0.000
F1 (IMP)	0.377	0.461	0.377
Training time (s)	57.672	61.137	58.640
Inference latency (ms)	117.194	88.650	87.305

Directed-communication and typed/relational edge variants are not implemented in this codebase. The GNN backbones used (GCN, GraphSAGE, and GAT) employ undirected, untyped message passing

5.6 Evaluation metrics

Detection metrics For stage-wise detection, we report per-class Precision (P), Recall (R), and F1, and macro-F1 across the four classes. Macro-F1 is treated as primary due to class imbalance.

Forecasting evaluation The forecasting architecture defines a configurable lead time and prediction horizon, but no empirical forecasting metrics are reported in the revised study. Under the strict leakage-controlled sequence-construction protocol, neither benchmark provides a sufficient set of valid longitudinal pre-impact instances. We therefore do not claim that a particular horizon, including $H = 30\Delta t$, is operationally optimal. Horizon and lead-time sensitivity remain to be evaluated on datasets containing sufficiently dense and reliable asset-level attack progression.

5.7 Baselines and ablations

We compare against representative baselines across tabular, graph, and temporal learning, and simple SOC-style heuristics. All baselines follow the same asset-disjoint protocol (Sect. 5.3); preprocessing is fitted on training identities only, and impact prediction uses strictly pre-cutoff inputs ($< t$) (Sect. 5.3).

Stage-wise detection baselines (four-class)
We report: (i) RF (feature-only), (ii) GCN (graph-only), and (iii) late fusion (probability averaging). We additionally include a strong tabular baseline (XGBoost/LightGBM) trained on $\mathbf{x}_i$ and a no-graph temporal baseline (per-asset GRU/LSTM over window sequences) to isolate the benefit of relational context.

Table 12 Held-out Macro-F1 obtained after validation-based baseline hyperparameter selection. These are controlled single-run tuning results and are separate from the three-seed main comparison

Dataset	Model	Test Macro-F1
ToN-IoT	RF	0.459
	XGBoost	0.371
	LightGBM	0.534
	GCN	0.654
	GraphSAGE	0.687
	GAT	**0.734**
Edge-IIoTset	**RF**	**0.6183**
	XGBoost	0.5931
	LightGBM	0.5655
	GCN	0.1847
	GraphSAGE	0.2987
	GAT	0.5304

Bold values indicate the highest performance for each evaluation metric

Graph learning baselines
To contextualise the GCN choice, we include GraphSAGE and GAT under the same node features and edge construction (Sect. 5.4).

Fair baseline tuning
To ensure a comparable evaluation, the principal tabular and graph baselines are selected using training/validation data only. Preprocessing, model selection, and hyperparameter tuning are performed without access to the held-out test identities. We apply the same model-selection criterion across the competing methods and report the resulting test performance irrespective of whether the proposed fusion model outperforms the individual baselines. This avoids preferential tuning of the RF–GCN configuration.

Table 12 reports the held-out Macro-F1 obtained after validation-based hyperparameter selection for the principal baselines on both datasets.

These single-seed tuning results are used to document model selection and should not be interpreted as the final repeated-run model comparison, which is reported separately in Sect. 6.1.1.

The tuning results confirm that the principal baselines were subjected to validation-based model selection. On ToN-IoT, GAT achieves the strongest held-out Macro-F1 (0.734) after validation-based tuning, followed by GraphSAGE (0.687) and GCN (0.654). On Edge-IIoTset, RF obtains the strongest held-out Macro-F1 (0.6183). These single-seed tuning results are reported as a model-selection check; the principal architectural conclusions are based on the three-seed evaluation reported in Sect. 6.1.1.

Table 13 Controlled meta-learner ablation for RF–GCN stacking (seed 42)

Dataset	Meta-learner	Macro-F1	Brier score
ToN-IoT	Logistic regression	0.308	0.080
	MLP	0.316	0.025
	Gradient boosting	**0.468**	**0.021**
Edge-IIoTset	Logistic regression	0.1153	0.4218
	MLP	0.1874	0.3953
	Gradient boosting	**0.3075**	**0.2531**

Bold values indicate the highest performance for each evaluation metric

Meta-learner sensitivity
The default RF–GCN stack uses logistic regression over out-of-fold base-model probabilities. We additionally evaluate an MLP and a gradient-boosted meta-learner while holding the RF and GCN base predictions fixed. The corrected experiments show that meta-learner choice materially affects stacking performance. On ToN-IoT, gradient boosting raises Macro-F1 from 0.308 with logistic regression to 0.468. On Edge-IIoTset, gradient boosting similarly outperforms logistic regression and the MLP. These results suggest that the linear logistic meta-learner is an important contributor to the observed stacking shortfall, although even the stronger meta-learner does not outperform the strongest standalone classifier on either benchmark.

Ablations
The revised evaluation includes stage-mapping sensitivity, graph-construction ablation, window-size sensitivity where valid, and stacking meta-learner ablation. Impact-forecasting ablations are not reported because the benchmark trajectories do not support a statistically valid forecasting evaluation.

5.8 Implementation details and hyperparameters

All preprocessing steps (encoding, standardisation, and feature selection) are fitted on the training split only and applied unchanged to the test split. The Random Forest uses class-weighting and tunes the number of trees and maximum depth on a validation set drawn from training identities only. The GCN is trained with cross-entropy and early stopping on validation identities, tuning the number of layers, hidden dimension, dropout, and learning rate. The forecasting architecture and its leakage-control rules are implemented and validated at the data-construction level, but no forecasting model performance is reported because the final benchmark audit did not produce sufficient valid longitudinal pre-impact training and test instances.

Class imbalance handling
Stage distributions are imbalanced in both benchmarks. Neural stage classifiers therefore use class-aware loss weighting, while tree-based models use corresponding balanced weighting where supported. Class weights are calculated from the training partition only. Because the revised study does not report empirical impact-forecasting performance, no impact-specific positive-class weighting or rare-event forecasting metric is included in the final results.

Computational cost and inference latency
Computational measurements are taken from the corrected real-data experiments. For ToN-IoT, graph construction and feature aggregation for the complete 34,435-node representation complete in under 15 s on the experimental CPU platform. The measured end-to-end scoring path requires approximately 185.5 ms per instance, corresponding to approximately 5.4 instances/s under the evaluated implementation. This value includes feature processing and the RF–GCN stacking path and should be interpreted as an implementation-level benchmark rather than a hard real-time deployment guarantee. Production latency will additionally depend on telemetry ingestion, online graph maintenance, batching, and deployment hardware. Edge-IIoTset graph-model inference measurements range from approximately 87.3-−117.2 ms under the evaluated configurations.

5.9 Statistical protocol and robustness

Repeated runs
The final stage-detection experiments are evaluated across three random seeds (42, 43, and 44) where stochastic training applies. We report mean±standard deviation across repeated runs. Deterministic estimators may consequently exhibit negligible or zero seed variation.

Bootstrap confidence intervals
We additionally use non-parametric bootstrap resampling with $B = 1000$ replicates to quantify test-set uncertainty for the stage-detection results. Where stable asset identities are available, resampling preserves the identity grouping rather than treating correlated observations as completely independent.

Validation safeguards
Before generating the final results, automated checks verify training-only preprocessing, absence of prohibited cross-split identity leakage, exclusion of synthetic smoke-test data, and dataset/configuration provenance. The final validation audit passed all implemented checks for both datasets.

The corrected ToN-IoT main comparison is performed on a fixed asset-disjoint split across three model seeds (42, 43, and 44). GAT, the strongest model, obtains Macro-F1 0.687 ± 0.041, while the logistic-regression Stacked RF–GCN obtains 0.395 ± 0.067. For the stacked model, an asset-level non-parametric bootstrap with $B = 1000$ valid resamples yields a 95% Macro-F1 confidence interval of [0.230, 0.408]. We do not report forecasting confidence inter-

Table 14 Stage-detection performance on the corrected ToN-IoT asset-disjoint test split. Results are mean±standard deviation across seeds 42, 43, and 44

Model	Macro-F1	F1(Benign)	F1(IAD)	F1(LMEP)	F1(IMP)
GAT	**0.687 ± 0.041**	0.990 ± 0.002	0.653 ± 0.010	0.803 ± 0.042	0.304 ± 0.124
GraphSAGE	0.671 ± 0.015	0.990 ± 0.000	0.631 ± 0.007	0.848 ± 0.008	0.214 ± 0.066
GCN	0.660 ± 0.039	0.991 ± 0.002	0.747 ± 0.054	0.717 ± 0.137	0.185 ± 0.020
Late fusion	0.658 ± 0.046	0.984 ± 0.004	0.590 ± 0.025	0.617 ± 0.178	0.439 ± 0.028
LightGBM	0.535 ± 0.002	0.979 ± 0.000	0.104 ± 0.002	0.596 ± 0.009	0.459 ± 0.000
RF	0.455 ± 0.003	0.973 ± 0.000	0.078 ± 0.000	0.315 ± 0.011	0.452 ± 0.000
GRU (sequence baseline)	0.430 ± 0.012	0.987 ± 0.003	0.000 ± 0.000	0.734 ± 0.045	0.000 ± 0.000
Stacked RF–GCN	0.395 ± 0.067	0.953 ± 0.037	0.000 ± 0.000	0.450 ± 0.226	0.177 ± 0.087
XGBoost	0.378 ± 0.006	0.976 ± 0.000	0.027 ± 0.019	0.507 ± 0.005	0.000 ± 0.000

Bold values indicate the highest performance for each evaluation metric

vals because no positive forecasting instances are present in validation or test.

6 Results and discussion

This section reports the validated real-data stage-detection experiments on ToN-IoT and Edge-IIoTset. We first compare feature-based, graph-based, and fusion-based classifiers and then examine stage-specific alerting, cross-dataset behaviour, modelling sensitivity, and robustness. Finally, we discuss the feasibility of leakage-controlled pre-impact forecasting and explain why the available benchmark representations do not support a statistically meaningful empirical forecasting evaluation.

6.1 Stage-wise detection performance

6.1.1 Four-class stage classification (primary results)

Evaluation protocol and baselines We report four-class event classification on held-out *asset-disjoint* test identities with labels {Benign, IAD, LMEP, IMP}. We compare: (i) RF (feature-only), (ii) GCN (graph-only), (iii) late fusion by probability averaging, and (iv) the proposed stacked RF–GCN model using a logistic-regression meta-learner trained on out-of-fold predictions from disjoint meta-train identities (Sect. 4.4.3). Because the stage distribution is imbalanced, we treat Macro-F1 as the primary metric and also report per-class F1.

Stronger baselines In addition to RF/GCN and late fusion, we report results for gradient-boosted trees (XGBoost/LightGBM) and for a no-graph temporal GRU/LSTM detector that models per-asset window sequences. For graph learning, we include GraphSAGE and GAT under the same node features and edge construction. These baselines help establish that improvements are not limited to comparisons against weaker references and clarify the benefit of relational (cross-asset) context beyond temporal smoothing alone.

ToN-IoT main results Table 14 reports the corrected stage-detection results across seeds 42, 43, and 44. GAT achieves the strongest Macro-F1 (0.687 ± 0.041), followed by GraphSAGE (0.671 ± 0.015), GCN (0.660 ± 0.039), and late fusion (0.658 ± 0.046). These results indicate that graph-based representations provide the strongest overall performance on the corrected ToN-IoT asset–window graph.

The logistic-regression RF–GCN stack reaches 0.395 ± 0.067 and does not outperform the individual graph learners or simple late fusion. A separate meta-learner ablation shows that gradient-boosting fusion increases stacked Macro-F1 to 0.468, indicating that meta-learner choice is an important contributor to this result. Nevertheless, the strongest standalone graph models remain clearly superior under the evaluated configuration.

Caveat on the GRU sequence baseline The GRU baseline consumes causal per-asset window prefixes and has access to genuine temporal sequences for some assets. However, temporal support is highly uneven under the asset-disjoint split. In training, 74.7% of evaluated instances have sequence length greater than one, whereas only 39.2% of validation instances and 30.4% of test instances have sequence length greater than one; the median validation and test sequence length is one. The held-out GRU result should therefore be interpreted as a sequence-model baseline rather than as a clean estimate of the benefit of long-horizon temporal reasoning.

6.1.2 Stage-specific one-vs-rest diagnostics

LMEP achieves the strongest one-vs-rest F1 (0.570), followed by IMP (0.519). IAD remains particularly difficult:

Table 15 One-vs-rest diagnostics on the corrected ToN-IoT test partition ($n = 3,313$)

Stage	# Pos.	Precision	Recall	F1	PR-AUC
IAD	48	0.005	0.042	0.009	0.008
LMEP	116	0.810	0.440	0.570	0.539
IMP	33	0.667	0.424	0.519	0.445

despite 48 positive test instances, precision is only 0.005 and F1 is 0.009. This is therefore no longer a single-example support artefact; rather, it indicates substantial overlap between IAD-like activity and benign traffic under the selected operating threshold (Table 15).

Given the very small number of positive observations, we do not draw quantitative conclusions from either the threshold-dependent metrics or the PR-AUC values in this diagnostic. The primary evidence for stage-detection performance therefore comes from the four-class evaluation and, importantly, from the substantially larger Edge-IIoTset external-validation experiment.

6.2 Cross-dataset validation on Edge-IIoTset

To examine whether the stage-aware formulation is specific to ToN-IoT, we repeat the four-class detection evaluation on Edge-IIoTset using the same Benign/IAD/LMEP/IMP abstraction and leakage-aware train/test processing. The Edge-IIoTset experiments provide an independent test of the portability of the proposed stage representation under a substantially different class distribution and network environment.

Edge-IIoTset results. Table 16 reports the external-validation results on Edge-IIoTset. In contrast to ToN-IoT, the strongest overall performance is obtained by the feature-based Random Forest, which achieves a Macro-F1 of 0.6146 ± 0.0027. XGBoost (0.5924 ± 0.0011) and LightGBM (0.5823 ± 0.0032) provide the next strongest results, followed by late fusion at 0.5213 ± 0.0473.

The graph-based models perform less consistently on this dataset. GraphSAGE achieves a Macro-F1 of 0.3816 ± 0.0576, GCN obtains 0.3481 ± 0.1361, and GAT obtains 0.3355 ± 0.1405. Nevertheless, GraphSAGE achieves the strongest IAD F1 (0.4674 ± 0.0135), indicating that relational representations can still provide stage-specific benefits even when they do not maximise aggregate Macro-F1.

The logistic-regression RF–GCN stack performs poorly (0.1647 ± 0.0604), substantially below both RF and late fusion. This confirms that learned probability stacking is not inherently beneficial across datasets and is sensitive to the calibration and distribution of the constituent model outputs. Together with the ToN-IoT results, the Edge-IIoTset evaluation demonstrates that the relative value of feature, graph, and fusion models is strongly dataset dependent.

6.2.1 Cross-dataset error analysis of RF–GCN stacking

Figure 3 provides a complementary error analysis of the default logistic-regression RF–GCN stack on the held-out test partitions of ToN-IoT and Edge-IIoTset. The matrices help explain why stacking does not outperform the strongest standalone models in either benchmark.

On ToN-IoT, the stacked model identifies a substantial fraction of LMEP instances correctly, but fails to recover any IAD instances and detects only a small fraction of IMP observations. In particular, all 48 IAD test instances are classified as Benign, while only 5 of 33 IMP instances are correctly classified as IMP. The dominant error pattern therefore reflects difficulty separating early-stage activity from benign behaviour and distinguishing impact from earlier attack stages.

The failure mode is more pronounced on Edge-IIoTset. The stacked model misclassifies 98% of Benign observations as LMEP and maps large fractions of IAD, LMEP, and IMP observations to the Benign class. This behaviour is consistent with the low aggregate Macro-F1 reported for stacking on Edge-IIoTset and suggests that the linear meta-learner does not combine the RF and GCN probability distributions robustly under the substantial cross-dataset shift. The meta-learner ablation in Table 13 supports this interpretation: gradient boosting improves stacked performance on both datasets, although it still does not surpass the strongest standalone classifier.

6.3 Feasibility of leakage-controlled pre-impact forecasting

The framework defines a leakage-controlled asset-level forecasting task in which only IAD/LMEP evidence observed before an asset-specific cutoff may be used to predict subsequent IMP. Following correction of temporal windowing, ToN-IoT now contains genuine longitudinal trajectories and 21,313 candidate pre-impact forecasting instances.

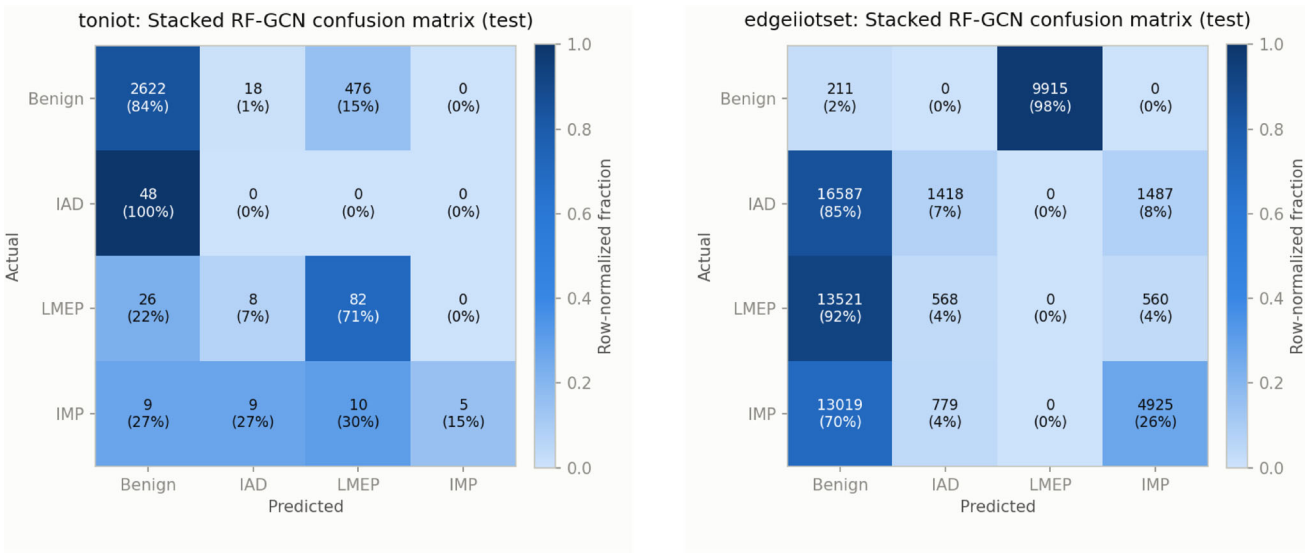
However, only five of these instances are positive. All five occur in the training partition (5/19,124), while validation contains zero positives (0/1,391) and test contains zero positives (0/798). Consequently, ROC-AUC, PR-AUC, thresholded F1, and Capture@$k\%$ cannot be meaningfully estimated on held-out data because both classes are not represented.

The underlying limitation reflects the attack structure of the benchmark. Of the 11 assets that ever reach IMP, seven have IMP as their first observed window and therefore provide no preceding IAD/LMEP history from which an impact forecast could be made. Only four IMP-reaching assets con-

Table 16 Stage-detection performance on Edge-IIoTset across three random seeds. Macro-F1 is the primary metric. The temporal GRU baseline is not evaluated because reliable longitudinal timestamps cannot be reconstructed from the selected Edge-IIoTset ML representation

Model	Macro-F1	F1(Benign)	F1(IAD)	F1(LMEP)	F1(IMP)
RF (feature-only)	**0.6146 ± 0.0027**	1.0000 ± 0.0000	0.4244 ± 0.0067	0.5027 ± 0.0036	0.5313 ± 0.0047
GCN (graph-only)	0.3481 ± 0.1361	0.4439 ± 0.2724	0.2511 ± 0.2048	0.3151 ± 0.0690	0.3821 ± 0.0103
Late fusion (RF+GCN)	0.5213 ± 0.0473	0.8520 ± 0.1099	0.3658 ± 0.0410	0.3680 ± 0.0830	0.4992 ± 0.0192
Stacked RF–GCN	0.1647 ± 0.0604	0.0456 ± 0.0588	0.1703 ± 0.1594	0.0000 ± 0.0000	0.4430 ± 0.0437
XGBoost	0.5924 ± 0.0011	1.0000 ± 0.0000	0.3838 ± 0.0036	0.4889 ± 0.0005	0.4970 ± 0.0010
LightGBM	0.5823 ± 0.0032	1.0000 ± 0.0000	0.4157 ± 0.0196	0.4613 ± 0.0037	0.4520 ± 0.0040
GraphSAGE	0.3816 ± 0.0576	0.3224 ± 0.2509	**0.4674 ± 0.0135**	0.3476 ± 0.0458	0.3889 ± 0.0998
GAT	0.3355 ± 0.1405	0.3382 ± 0.3087	0.3127 ± 0.1553	0.3267 ± 0.0876	0.3642 ± 0.0783

Bold values indicate the highest performance for each evaluation metric



(a) ToN-IoT (b) Edge-IIoTset

Fig. 3 Confusion matrices for the default logistic-regression RF–GCN stacked classifier on the held-out test partitions: (**a**) corrected ToN-IoT ($n = 3,313$) and (**b**) Edge-IIoTset ($n = 62,990$). Cells report raw counts together with row-normalised percentages. The figure illustrates substantially different failure modes across the two benchmarks: ToN-IoT errors concentrate on IAD and IMP, whereas Edge-IIoTset exhibits pronounced cross-class collapse, particularly toward the Benign class. These results reinforce that simple probability stacking does not robustly exploit RF and GCN complementarity across datasets

tain genuine pre-impact observations, and the valid positive forecasting instances associated with these trajectories fall within the training partition under the fixed asset-disjoint split.

We deliberately do not redistribute these assets across partitions merely to create reportable forecasting metrics, because doing so after observing the outcome would compromise the identity-preserving evaluation protocol. Accordingly, no empirical forecasting-performance claim is made. The dual-stream GRU is retained as a leakage-controlled methodological formulation whose predictive performance requires future evaluation on datasets containing a larger number of independent longitudinal escalation episodes.

6.4 Robustness and uncertainty

The stage-detection experiments are repeated across three random seeds where stochastic training applies. We report mean and standard deviation across these runs and additionally use non-parametric bootstrap confidence intervals to characterise sampling uncertainty. The revised analysis does not assume that small differences in Macro-F1 imply statistically meaningful model superiority. Instead, the cross-dataset results show that the relative effectiveness of feature-based, graph-based, and fusion models depends on the dataset and graph representation.

7 Reproducibility and data-correction statement

During revision, we identified a timestamp-resolution defect in the ToN-IoT temporal-windowing implementation. The original windowing function assumed nanosecond-resolution pandas timestamps and consequently collapsed all ToN-IoT records into a single temporal window identifier under the software environment used for the revised experiments. The function was corrected to be independent of datetime resolution and verified using regression tests covering second-, millisecond-, microsecond-, and nanosecond-resolution timestamps.

All ToN-IoT experiments reported in this revised manuscript were rerun after the correction. The corrected representation contains 34,435 asset–window instances, including 86 IAD, 298 LMEP, and 243 IMP instances, together with 22,899 genuine temporal-continuity edges. All pre-correction outputs were excluded from the reported results. Edge-IIoTset is unaffected because its validated record-level processing path does not use the affected temporal-windowing function. The corrected code, experiment configurations, and audit artefacts are included in the released repository.

8 Conclusion

This paper investigated an ATT&CK-inspired stage-aware intrusion-detection framework for IIoT cybersecurity and evaluated the Benign/IAD/LMEP/IMP abstraction on two independently collected benchmarks, ToN-IoT and Edge-IIoTset. The experiments compare feature-based, graph-based, and fusion-based classifiers under leakage-aware evaluation and examine sensitivity to stage mapping, graph construction, window size, meta-learner choice, model tuning, and statistical uncertainty.

On the corrected ToN-IoT representation, graph-based models provide the strongest stage-detection performance. GAT achieves Macro-F1 0.687 ± 0.041, followed by GraphSAGE (0.671 ± 0.015) and GCN (0.660 ± 0.039). In contrast, the default logistic-regression RF–GCN stack obtains 0.395 ± 0.067. A controlled meta-learner ablation raises stacked Macro-F1 to 0.468 with gradient boosting, indicating that fusion effectiveness depends strongly on the meta-learner; however, the strongest standalone graph learners remain superior under the evaluated configurations. On Edge-IIoTset, RF performs best (0.615 Macro-F1), reinforcing the conclusion that the relative value of feature-based and relational representations is dataset dependent.

We additionally formulate a strict leakage-controlled pre-impact forecasting protocol. Correct temporal reconstruction produces 21,313 candidate ToN-IoT forecasting instances, but only five are positive and all occur in the training partition, leaving no positive validation or test examples. We therefore make no empirical forecasting-performance claim. This result highlights a distinction between defining a leakage-safe forecasting methodology and having a benchmark with sufficient independent longitudinal escalation episodes to evaluate it statistically.

Limitations and deployment scope ToN-IoT and Edge-IIoTset are controlled IoT/IIoT cybersecurity testbeds rather than complete multi-organisational industrial supply chains. They do not fully represent supplier trust relationships, third-party remote access, cloud dependencies, heterogeneous administrative domains, or operational concept drift. Furthermore, asset-disjoint evaluation tests generalisation to unseen identities but does not fully reproduce streaming cold-start deployment, where new assets and communication edges appear incrementally. Future work should therefore evaluate online graph updates, progressive cold-start behaviour, richer directed and typed graph models, and longitudinal attack-progression datasets.

Although the corrected ToN-IoT representation provides substantially stronger window-level minority-stage support, malicious behaviour remains concentrated in a small number of source assets; conclusions concerning generalisation to previously unseen malicious identities should therefore remain cautious.

Author Contributions WGA: system modelling, design, security analysis, writing – original draft. GE: supervision, review. HSL: supervision, review. CM: supervision, review.

Funding This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Data Availability No datasets were generated or analysed during the current study.

Code Availability The implementation used for the revised experiments will be made publicly available to support reproducibility. The repository includes dataset adapters, stage-mapping configurations, leakage-aware preprocessing, identity-preserving splitting, graph construction, baseline models, stacking and ablation experiments, sensitivity analyses, bootstrap evaluation, inference-latency measurement, and scripts for reproducing the reported tables and figures. The original ToN-IoT and Edge-IIoTset datasets are not redistributed and should be obtained from their respective public sources. Code repository: https://github.com/AfrifaW/Multi-Stage-Attack-Detection

Declarations

Conflict of interest The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the

source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

1. Huang, X., et al.: Securing the internet of things: supply chain vulnerabilities and threat mitigation. IEEE Commun. Surv. Tutorials **23**(2), 1027–1051 (2021). https://doi.org/10.1109/COMST.2021.3051234
2. Alotaibi, B.: A survey on industrial internet of things security. Sensors **23**(17), 7470 (2023). https://doi.org/10.3390/s23177470
3. Huang, X., et al.: Securing the internet of things: supply chain vulnerabilities and threat mitigation. IEEE Commun. Surv. Tutorials **23**(2), 1027–1051 (2021). https://doi.org/10.1109/COMST.2021.3051234
4. Sallam, M.E., Rashid, S., Ahmed, S.H.: Internet of things adoption in supply chain management: a review and future research directions. Ain Shams Eng. J. **14**(6), 102244 (2023). https://doi.org/10.1016/j.asej.2023.102244
5. IoT Security Foundation: Securing the Internet of Things Supply Chain. Whitepaper. https://iotsecurityfoundation.org Accessed 2025 (2022)
6. Device Authority: Industrial IoT Security Threats: Top Risks and Mitigation Strategies 2025. Technical whitepaper. https://deviceauthority.com Accessed 2025 (2025)
7. Kumar, M., Epiphaniou, G., Maple, C.: Security of cyber-physical additive manufacturing supply chain: survey, attack taxonomy and solutions. Comput. Secur. 104557 (2025)
8. Gyamfi, E., Jurcut, A.: Intrusion detection in internet of things systems: A review on design approaches leveraging multi-access edge computing, machine learning, and datasets. Sensors **22**(10), 3744 (2022). https://doi.org/10.3390/s22103744
9. Villafranca, A., Thant, K.M., Tasic, I., Cano, M.-D.: Ai-enabled iot intrusion detection: Unified conceptual framework and research roadmap. Mach. Learn. Knowl. Extraction **7**(4), 115 (2025). https://doi.org/10.3390/make7040115
10. Lightbody, D., Ngo, D.-M., Temko, A., Murphy, C.C., Popovici, E.: Dragon_pi: Iot side-channel power data intrusion detection dataset and unsupervised convolutional autoencoder for intrusion detection. Fut. Internet **16**(3), 88 (2024). https://doi.org/10.3390/fi16030088
11. MITRE Corporation: MITRE ATT&CK®: Adversarial Tactics, Techniques, and Common Knowledge. https://attack.mitre.org. Accessed 2025 (2015)
12. Alexander, O., Belisle, M., Steele, J.: Mitre att&ck for industrial control systems: design and philosophy. Technical Report 19-03307-03, MITRE Corporation, Bedford, MA, USA (2020). https://attack.mitre.org
13. Rani, R., Kumar, M., Epiphaniou, G., Maple, C.: Icsthreatqa: A knowledge-graph enhanced question answering model for industrial control system threat intelligence. Expert Syst. Appl., 130180 (2025)
14. Roy, S., Panaousis, E., Noakes, C., Laszka, A., Panda, S., Loukas, G.: Sok: The mitre att&ck framework in research and practice. arXiv preprint arXiv:2304.07411 (2023) https://doi.org/10.48550/arXiv.2304.07411
15. Al-Sada, B., Sadighian, A., Oligeri, G.: Mitre att&ck: State of the art and way forward. arXiv preprint arXiv:2308.14016 (2023) https://doi.org/10.48550/arXiv.2308.14016
16. Sun, Z., Teixeira, A., Toor, S.: Gnn-ids: Graph neural network based intrusion detection system. In: Proceedings of the 19th International Conference on Availability, Reliability and Security (ARES). ACM, Vienna, Austria (2024). https://doi.org/10.1145/3664476.3664515
17. Wang, Y., Han, Z., Du, Y., Li, J., He, X.: Bs-gat: A network intrusion detection system based on graph neural network for edge computing. Cybersecurity **8**(1), 1–18 (2025). https://doi.org/10.1186/s42400-024-00296-8
18. Dong, G., Tang, M., Wang, Z., et al.: Graph neural networks in iot: A survey. ACM Trans. Sensor Netw. **19**(2), 1–39 (2023). https://doi.org/10.1145/3565973
19. Zhong, M., Lin, M., Zhang, C., Xu, Z.: A survey on graph neural networks for intrusion detection systems: Methods, trends and challenges. Comput. Secur. **141**, 103821 (2024). https://doi.org/10.1016/j.cose.2024.103821
20. Ahanger, A.S., Khan, S.M., Masoodi, F., Salau, A.O.: Advanced intrusion detection in internet of things using graph attention networks. Sci. Rep. **15**(1), 9831 (2025). https://doi.org/10.1038/s41598-025-94624-8
21. Alsaedi, A., Moustafa, N., Tari, Z., Mahmood, A., Anwar, A.: Ton_iot telemetry dataset: A new generation dataset of iot and iiot for data-driven intrusion detection systems. IEEE Access **8**, 165130–165150 (2020). https://doi.org/10.1109/ACCESS.2020.3022862
22. Moustafa, N., Ahmed, M., Ahmed, S.: Data analytics-enabled intrusion detection: Evaluations of ton_iot linux datasets. In: 2020 IEEE 19th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom), pp. 727–734 (2020)
23. Moustafa, N., Keshk, M., Debie, E., Janicke, H.: Federated ton_iot windows datasets for evaluating ai-based security applications. In: 2020 IEEE 19th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom), pp. 848–855 (2020)
24. Moustafa, N., Ahmed, M., Ahmed, S.: Data analytics-enabled intrusion detection: Evaluations of ton_iot linux datasets. arXiv preprint (2020) arXiv:2010.08521
25. Kumar, M., Epiphaniou, G., Maple, C.: Comprehensive threat analysis in additive manufacturing supply chain: a hybrid qualitative and quantitative risk assessment framework. Prod. Eng. Res. Devel. **18**(6), 955–973 (2024)
26. Roesch, M.: Snort: Lightweight intrusion detection for networks. In: Proceedings of the 13th USENIX Conference on System Administration (LISA), pp. 229–238. (1999)
27. Open Information Security Foundation: Suricata: Open Source IDS/IPS/NSM Engine. https://suricata.io/. Accessed: 2025-11-26
28. Alotaibi, B.: A survey on industrial internet of things security: Requirements, attacks, AI-based solutions, and edge computing opportunities. Sensors **23**(17), 7470 (2023). https://doi.org/10.3390/s23177470
29. Rahman, M.M., Shakil, S.A., Mustakim, M.R.: A survey on intrusion detection system in IoT networks. Cyber Secur. Appl. **3**, 100082 (2025). https://doi.org/10.1016/j.csa.2024.100082
30. Anwer, R.W.: Advanced intrusion detection in the industrial internet of things: A comprehensive survey. Comput. Netw. **238**, 110090 (2025). https://doi.org/10.1016/j.comnet.2024.110090
31. Jawad, M.A., et al.: A survey of network intrusion detection systems for industrial internet of things scenarios. Ad Hoc Netw. **158**, 103274 (2025). https://doi.org/10.1016/j.adhoc.2024.103274
32. Mohy-eddine, M., Guezzaz, A., Benkirane, S., Azrour, M.: An effective intrusion detection approach based on ensemble learning

for IIoT edge computing. J. Comput. Virol. Hack. Tech. **19**(4), 469–481 (2023). https://doi.org/10.1007/s11416-023-00456-9
33. Nuaimi, M., Fourati, L.C.: Ensemble learning approach for intrusion detection systems in industrial internet of things. In: 2023 20th ACS/IEEE International Conference on Computer Systems and Applications (AICCSA), pp. 1–8 (2023). https://doi.org/10.1109/AICCSA59173.2023.10479270
34. Almotairi, A., et al.: Enhancing intrusion detection in IoT networks using a heterogeneous machine-learning-based stack classifier model. Int. J. Distrib. Sens. Netw. **20**(3), 1–18 (2024). https://doi.org/10.1177/15501477241234567
35. Golchha, R.: Voting-based ensemble learning approach for cyber intrusion detection in big data IIoT environments. Procedia Comput. Sci. **218**, 1566–1575 (2023). https://doi.org/10.1016/j.procs.2023.11.250
36. Fraihat, S., et al.: Intrusion detection system for large-scale IoT netflow-based networks. Array **21**, 100234 (2023). https://doi.org/10.1016/j.array.2023.100234
37. Nandanwar, H., Katarya, R.: Deep learning enabled intrusion detection system for industrial IoT environment. Expert Syst. Appl. **249**, 123808 (2024). https://doi.org/10.1016/j.eswa.2024.123808
38. Hossain, M.A., et al.: Deep learning-based intrusion detection for IoT networks. EURASIP J. Inf. Secur. **2025**(4), 1–23 (2025). https://doi.org/10.1186/s13635-025-00202-w
39. Gaber, T.: Industrial internet of things intrusion detection method based on particle swarm optimization and bat algorithm. Secur. Commun. Netw. (2023). https://doi.org/10.1155/2023/3939895
40. Gueriani, A., Kheddar, H., Mazari, A.C.: Deep reinforcement learning for intrusion detection in IoT: A survey. In: 2023 2nd International Conference on Electronics, Energy and Measurement (IC2EM), pp. 1–6 (2024). https://doi.org/10.1109/IC2EM58549.2023.10460745
41. Yu, S., Zhai, R., Shen, Y., Wu, G., Zhang, H., Yu, S., Shen, S.: Deep q-network-based open-set intrusion detection solution for industrial internet of things. IEEE Internet Things J. **11**(7), 12536–12550 (2024). https://doi.org/10.1109/JIOT.2023.3333903
42. Shen, S., Cai, C., Li, Z., Shen, Y., Wu, G., Yu, S.: Deep q-network-based heuristic intrusion detection against edge-based siot zero-day attacks. Appl. Soft Comput. **150**, 111080 (2024). https://doi.org/10.1016/j.asoc.2023.111080
43. Yu, S., Wang, X., Shen, Y., Wu, G., Yu, S., Shen, S.: Novel intrusion detection strategies with optimal hyper parameters for industrial internet of things based on stochastic games and double deep q-networks. IEEE Internet Things J. **11**(17), 29132–29145 (2024). https://doi.org/10.1109/JIOT.2024.3406386
44. Shen, S., Niu, J., Shen, Y., Dong, J., Ke, W., Wang, T., Li, R.: MT-KD: Multi-teacher knowledge distillation for privacy-aware unlearning in iiot intrusion detection systems. IEEE Trans. Cognit. Commun. Netw. **12**, 6344–6357 (2026). https://doi.org/10.1109/TCCN.2026.3665917
45. Wang, L., Jajodia, S., Singhal, A.: Using attack graphs for correlating, hypothesizing, and predicting intrusion alerts. Comput. Commun. **29**(15), 2917–2933 (2006). https://doi.org/10.1016/j.comcom.2006.04.001
46. Ning, P., Xu, D.: Learning attack strategies from intrusion alerts. In: Proceedings of the 10th ACM Conference on Computer and Communications Security (CCS), pp. 200–209 (2003). https://doi.org/10.1145/948109.948138
47. Roschke, S., Cheng, F., Meinel, C.: A new alert correlation algorithm based on attack graph. In: Proceedings of the 4th International Conference on Computational Intelligence in Security for Information Systems. CISIS, pp. 58–67. Springer, (2011). https://doi.org/10.1007/978-3-642-21424-0_8
48. Holgado, P., Villagrá, V.A., Vázquez, L.: Real-time multistep attack prediction based on hidden markov models. IEEE Trans. Dependable Secure Comput. **17**(1), 134–147 (2020). https://doi.org/10.1109/TDSC.2017.2719684
49. Ghafir, I., Kyriakopoulos, K.G., Li, S.: Hidden markov models and alert correlations for the prediction of advanced persistent threats. IEEE Access **7**, 99508–99520 (2019). https://doi.org/10.1109/ACCESS.2019.2930200
50. Cheng, Q., Wu, C., Zhou, S.: Discovering attack scenarios via intrusion alert correlation using graph convolutional networks. IEEE Commun. Lett. **25**(5), 1564–1567 (2021). https://doi.org/10.1109/LCOMM.2021.3059625
51. Anjum, M.M., Iqbal, S., Hamelin, B.: Anubis: A provenance graph-based framework for advanced persistent threat detection. In: Proceedings of the 37th ACM/SIGAPP Symposium on Applied Computing (SAC), pp. 1642–1651 (2022). https://doi.org/10.1145/3477314.3507097
52. Wang, T., Li, Y., Zhang, W., et al.: Provenance graph-based deep learning framework for advanced persistent threat detection. Appl. Sci. **15**(16), 8833 (2025). https://doi.org/10.3390/app15168833
53. Husák, M., Janáček, A., et al.: Survey of attack projection, prediction, and forecasting in cyber security. IEEE Commun. Surv. Tutor. **21**(1), 640–660 (2019). https://doi.org/10.1109/COMST.2018.2871866
54. Sun, Y., Paul, S., et al.: Data-driven cybersecurity incident prediction: A survey. IEEE Commun. Surv. Tutor. **21**(2), 1744–1772 (2019). https://doi.org/10.1109/COMST.2018.2885561
55. Albasheer, H., Siraj, M.M., Mubarakali, A., et al.: Cyber-attack prediction based on network intrusion detection systems for alert correlation techniques: A survey. Sensors **22**(4), 1494 (2022). https://doi.org/10.3390/s22041494
56. Ansari, M.S., Bartoš, V., Lee, B.: Gru-based deep learning approach for network intrusion alert prediction. Futur. Gener. Comput. Syst. **128**, 235–247 (2022). https://doi.org/10.1016/j.future.2021.09.040
57. Sankar, A., Wu, Y., Gou, L., Zhang, W., Yang, H.: Dysat: Deep neural representation learning on dynamic graphs via self-attention networks. In: Proceedings of the 13th International Conference on Web Search and Data Mining (WSDM), pp. 519–527. ACM, (2020). https://doi.org/10.1145/3336191.3371845
58. Pareja, A., Domeniconi, G., Chen, J., Ma, T., Suzumura, T., Kanezashi, H., Kaler, T., Schardl, T.B., Leiserson, C.E.: Evolvegcn: Evolving graph convolutional networks for dynamic graphs. In: Proceedings of the Thirty-Fourth AAAI Conference on Artificial Intelligence (AAAI), vol. 34, pp. 5363–5370 (2020). https://doi.org/10.1609/aaai.v34i04.5984
59. Xu, D., Ruan, C., Korpeoglu, E., Kumar, S., Achan, K.: Inductive representation learning on temporal graphs. In: International Conference on Learning Representations (ICLR, (2020)
60. Rossi, E., Chamberlain, B., Frasca, F., Eynard, D., Monti, F., Bronstein, M.M.: Temporal graph networks for deep learning on dynamic graphs. arXiv preprint arXiv:2006.10637 (2020) arXiv:2006.10637 [cs.LG]
61. Kumar, M., Epiphaniou, G., Maple, C.: Leveraging semantic relationships to prioritise indicators of compromise in additive manufacturing systems. In: International Conference on Applied Cryptography and Network Security, pp. 321–335. Springer (2023)
62. Kumar, M., Epiphaniou, G., Maple, C.: A novel intelligence and information acquisition system for managing indicators of compromise in distributed responsive manufacturing systems. In: IET Conference Proceedings CP846, vol. 2023, pp. 144–150. (2023). (**IET**)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.